

國立臺灣大學電機資訊學院生醫電子與資訊學研究所

碩士論文(初稿)

Graduate Institute of Biomedical Electronics and Bioinformatics

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis (Draft)

基於基因體基礎模型之總體基因體短讀序列分類學分
類

Token-level Genomic Foundation Model Fine-tuning
for Metagenomic Taxonomic Classification

楊明儒

Ming-Ju Yang

指導教授: 劉子毓 博士

Advisor: Tzu-Yu Liu, Ph.D.

中華民國 115 年 9 月

September 2026





誌謝





基於基因體基礎模型之總體基因體短讀序列分類學分 類

研究生：楊明儒

指導教授：劉子毓 博士

國立臺灣大學 生醫電子與資訊學研究所

摘要

總體基因體學的分類學分類——即將每條定序短讀序列指派至正確的微生物分類單元——是微生物群落分析的核心計算挑戰。本論文系統性評估基因體基礎模型於總體基因體短讀序列分類的適用性，並以受控消融分離資料規模、骨幹深度、預訓練與 tokenization。我們以 Nucleotide Transformer v2 (498M 參數) 搭配低秩適應 (LoRA) 與注意力池化分類頭，作為一般用途基因體基礎模型的代表。在固定 NT-v2 非重疊 6-mer 架構下，訓練資料量主導準確度：屬層級 120 類由 500K 增至 5M 提升 7.76 個百分點 (55.29% 至 63.05%)，50M 達 67.07%；再擴 5× 至 250M 僅 +0.22 個百分點 (67.29%)。此飽和是 tokenizer 的上限而非資料的上限：同一批讀序、同一 50M → 250M 區間，重疊 13-mer 的 MetaTransformer 由 87.5% 續升至 98.7%。先前以 29 層預訓練 NT-v2 對單層隨機初始化 Transformer 得到的 +13.19 個百分點，把預訓練與深度混在一起。在相同 6-mer tokenizer、相同 50M 讀序、同一 clean_common 100K 測試檔與反向互補測試時增強 (RC TTA) 下，從頭訓練的 1 / 8 / 16 層模型分別為 53.92% / 65.44% / 67.94%，而 NT-v2 LoRA 為 67.08%；深度本身即解釋 +14.02 個百分點 (1→16 層)，預訓練淨貢獻約為 0 (16 層從頭訓練甚至高出 0.86 個百分點)。預訓練的可辯護效益出現在標籤稀缺時：同樣深度對齊的 16 層對照，5M 讀序上 NT-v2 為 63.05%、從頭訓練為 54.11% (+8.94 個百分點；該 63.05% 為內部驗證數字，見正文)。跨架構 tokenization 消融中，重疊 13-mer MetaTransformer 於相同 50M 讀

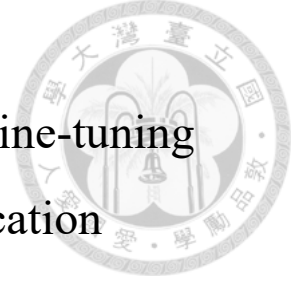


序達 87.42%，較 NT-v2 + LoRA (67.07%) 高 20.35 個百分點。同架構 16 層對照進一步顯示：hashed 13-mer (2 GiB) 達 83.83%，exact 13-mer 達 85.63%；6-mer 即使加到 29 層也僅 69.52%。物種層級 1,535 類上，per-genus 50M LoRA 於 oracle genus routing 達 29.5% Top-1，預測 routing 降至 15.6%；subgenus 路由為負結果。樣本層級在物種平衡隨機分割合成樣本中，67% 單讀序列準確度可轉為相對豐度 Pearson $r = 0.993$ ，但屬層級豐度無法用來選方法 (Kraken2+Bracken $r = 0.997$)，真實多樣性社群泛化仍待驗證。本研究範圍為一般用途基因體基礎模型 (NT-v2、DNABERT)；METAGENE-1 與 GenomeOcean 等以總體基因體預訓練的模型未經評估。結論是 tokenization 為主要瓶頸，深度可以彌補不良 tokenizer，預訓練提升的是資料效率而非 50M 上限。

關鍵字：

基因體基礎模型、總體基因體學、分類學分類、tokenization、低秩適應、注意力池化

Token-level Genomic Foundation Model Fine-tuning for Metagenomic Taxonomic Classification



Student: Ming-Ju Yang

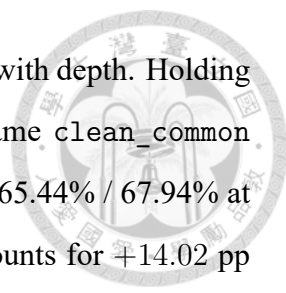
Advisor: Tzu-Yu Liu, Ph.D.

Graduate Institute of Biomedical Electronics and Bioinformatics
National Taiwan University

ABSTRACT

Metagenomic taxonomic classification—assigning each sequencing read to its source organism in the microbial taxonomy—is a fundamental computational challenge in microbiome analysis. This thesis evaluates genomic foundation models (GFM) for short-read taxonomic classification through controlled ablations of data scale, backbone depth, pre-training, and tokenization. We take Nucleotide Transformer v2 (NT-v2, 498M parameters) with Low-Rank Adaptation (LoRA) and an attention-pooling head as the representative general-purpose GFM.

Within the fixed NT-v2 non-overlapping 6-mer framework, training data volume dominates architecture and training tricks: a $10\times$ increase yields $+7.76$ percentage points (pp) in genus-level accuracy (55.29% to 63.05% on 120 genera at 5M reads), scaling further to 67.07% at 50M, whereas technique ablations move accuracy by at most ± 0.5 pp. A further $5\times$ increase to 250M yields only $+0.22$ pp (67.29%). That saturation is a property of the tokenizer, not of the data: trained on the same reads over the same 50M $\rightarrow$ 250M range, an overlapping 13-mer model scales from 87.5% to 98.7% genus Top-1. Reverse-complement test-time augmentation (RC TTA) improves accuracy by $+0.1$ to $+1.5$ pp. The previously reported $+13.19$ pp gap between 29-layer pre-trained NT-v2 and a one-



layer randomly initialized Transformer at 50M conflates pre-training with depth. Holding the 6-mer tokenizer, 50M reads, attention-pooling head, and the same `clean_common` 100K file fixed under RC TTA, from-scratch models score 53.92% / 65.44% / 67.94% at 1 / 8 / 16 layers, against 67.08% for NT-v2 LoRA: depth alone accounts for +14.02 pp (1 $\rightarrow$ 16 layers), and the net contribution of pre-training at this scale is ≈ 0 (the 16-layer from-scratch model is +0.86 pp higher). A defensible pre-training benefit appears when labels are scarce: at matched 16-layer depth and 6-mer tokenization, NT-v2 reaches 63.05% at 5M reads versus 54.11% from scratch (+8.94 pp; the 63.05% figure is the recorded internal-validation number, as disclosed in the main text).

Cross-architecture tokenization ablations remain decisive: MetaTransformer with overlapping 13-mer tokenization, trained from scratch on the same 50M reads, reaches 87.42% genus Top-1, +20.35 pp above NT-v2 + LoRA (67.07%). A same-architecture 16-layer comparison on `clean_common` RC TTA further shows that a hashed 13-mer (2 GiB embedding) reaches 83.83% and an exact 13-mer 85.63%, while a 29-layer from-scratch 6-mer reaches only 69.52%. At the species level (1,535 classes), per-genus 50M LoRA classifiers under oracle-genus routing reach 29.5% Top-1; predicted routing through the genus model lowers this to 15.6%. Subgenus routing is a negative result. In synthetic species-balanced random-partition samples, 67% per-read accuracy translates to Pearson $r = 0.993$ for relative abundance, but genus-rank abundance cannot choose a method (Kraken2+Bracken $r = 0.997$); evaluation on real and compositionally variable communities remains future work. The scope is general-purpose GFMs (NT-v2, DNABERT); metagenome-pre-trained models such as METAGENE-1 and GenomeOcean were not tested. Tokenization is the bottleneck; depth can compensate for a weak tokenizer; pre-training buys data efficiency, not a higher 50M ceiling.

Keywords:

Genomic Foundation Model, Metagenomics, Taxonomic Classification, Tokenization, LoRA, Attention Pooling

CONTENTS



誌謝

摘要

ABSTRACT

CONTENTS

LIST OF FIGURES

LIST OF TABLES

List of Symbols and Abbreviations

Chapter 1 Introduction

1.1	Background and Motivation	1
1.2	Problem Statement	3
1.3	Contributions	5
1.4	Thesis Organization	7

Chapter 2 Background and Related Work

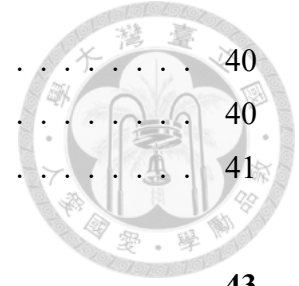
2.1	Metagenomic Taxonomic Classification	9
2.1.1	Alignment-Based Methods	9
2.1.2	k -mer-Based Methods	10
2.1.3	Deep Learning Methods	10
2.2	Genomic Foundation Models	12
2.2.1	Nucleotide Transformer	12
2.2.2	Other Genomic Foundation Models	13
2.2.3	GFM for Metagenomic Classification	14
2.3	Parameter-Efficient Fine-Tuning	16
2.3.1	Low-Rank Adaptation (LoRA)	16
2.3.2	Other PEFT Methods	17
2.4	Attention Mechanisms for Sequence Aggregation	17
2.4.1	Mean Pooling	18
2.4.2	Attention Pooling	18
2.4.3	Multi-Head Attention Pooling	19

2.5	Data Augmentation for DNA Sequences	19
2.5.1	Reverse-Complement Symmetry in DNA	19
2.5.2	Strategies for Exploiting RC Symmetry	20
2.6	Handling Class Imbalance	21
2.6.1	Loss-Level Approaches	21
2.6.2	Sampling-Level Approaches	22
2.7	Summary	22

Chapter 3 Methodology 23

3.1	System Overview	23
3.2	Backbone: Nucleotide Transformer v2	24
3.2.1	Tokenization	25
3.2.2	Encoder Architecture	26
3.2.3	LoRA Adaptation	27
3.2.4	Ablation: Shallow Transformer Backbone	27
3.2.5	Same-Architecture Tokenizer Variants	29
3.3	Classification Head: Attention Pooling	29
3.3.1	Architecture	30
3.3.2	Alternative Heads	31
3.4	Training Strategy	31
3.4.1	Two-Phase Training	31
3.4.2	Optimization	32
3.4.3	Transfer Learning Across Experiments	33
3.5	Data Pipeline	33
3.5.1	Reference Database and Taxonomy	33
3.5.2	ART Read Simulation	34
3.5.3	Balanced Subsampling	36
3.5.4	Reverse-Complement Augmentation	37
3.5.5	Data Splitting	37
3.6	Evaluation Methodology	38
3.6.1	Metrics	38
3.6.2	Reverse-Complement Test-Time Augmentation	39
3.6.3	Species-Level Evaluation Modes	39
3.7	Computational Considerations	40
3.7.1	Hardware	40

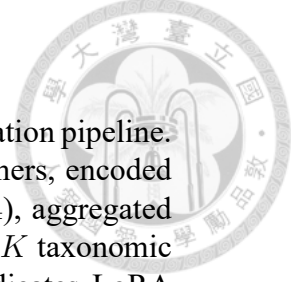
3.7.2	Memory Optimization	40
3.7.3	Resource Monitoring	40
3.8	Summary	41



Chapter 4	Experiments and Results	43
4.1	Experimental Setup	43
4.1.1	Dataset	43
4.1.2	Evaluation Protocol	44
4.1.3	Hardware and Training Infrastructure	45
4.2	Phase 1: Frozen Embedding Baselines	45
4.2.1	Setup	45
4.2.2	Results	45
4.3	Phase 2: Token-Level Classification with LoRA	46
4.3.1	Architecture and Training Techniques (v1–v7, 500K Dataset)	46
4.4	Data Scaling Experiments	48
4.4.1	NT-Genus-5M: 5M Balanced Reads	48
4.4.1.1	Setup	48
4.4.1.2	Results	48
4.4.1.3	Training Dynamics	49
4.4.2	NT-Genus Model: 50M Balanced Reads	50
4.4.2.1	Results	50
4.4.3	Data Scaling Analysis	52
4.4.4	Balancing Axis: Species-Balanced vs. Genus-Balanced Training	55
4.5	RC Test-Time Augmentation Analysis	57
4.6	Computational Resource Analysis	58
4.6.1	Observed Resource Usage	58
4.6.2	Bottleneck Analysis	58
4.7	Comparison with MetaTransformer	60
4.8	Backbone Ablation: Depth, Then Pre-training	62
4.8.1	Confounded baseline: 29-layer NT-v2 vs. one-layer from scratch	62
4.8.2	Depth sweep at matched 6-mer tokenization	63
4.8.3	Data-efficiency crossover at matched depth	64
4.9	Tokenization Ablation: MetaTransformer at 50M Scale	65
4.10	Species-Level Classification at Scale	75
4.10.1	NT-Species at 50M	75

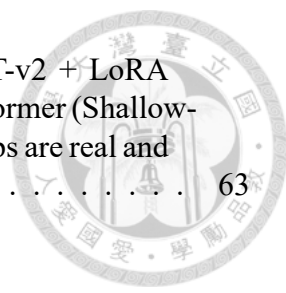
4.10.2	Independent Test Set and Data-Leakage Audit	75
4.10.3	Hierarchical Evaluation	76
4.10.4	Subgenus Routing: A Negative Result	78
4.11	Sample-Level Application Evaluation	78
4.11.1	Genus Abundance and Detection	79
4.11.2	Matched Protocol across Models	79
4.11.3	Species-Level Samples and Kraken2	80
4.12	Summary of Findings	82
Chapter 5	Conclusion and Future Work	85
5.1	Summary of Contributions	85
5.2	Limitations	88
5.3	Future Work	90
5.3.1	Microbial Foundation Models with Overlapping Long- k Tokenization	90
5.3.2	Streaming Infrastructure for Species-Level Scaling	91
5.3.3	Hierarchical Routing Improvements for Species Classification . . .	91
5.3.4	Real-World Validation and Calibrated Sample-Level Estimation . .	92
5.4	Concluding Remarks	93
	References	95

LIST OF FIGURES



- Figure 3.1 Overview of the proposed token-level GFM classification pipeline. A 150 bp DNA read is tokenized into non-overlapping 6-mers, encoded by the LoRA-adapted NT-v2 backbone (29 layers, $d=1024$), aggregated via multi-head attention pooling, and classified into one of K taxonomic classes (120 genera or 1,535 species). Orange badge indicates LoRA adapter modules; dashed boxes delineate the backbone and classification head parameter groups. 24
- Figure 4.1 Training dynamics for v3 (500K, left), NT-Genus-5M (5M, centre), and NT-Genus (50M, right). Dashed lines show training accuracy; solid lines show validation accuracy. The shaded region highlights the train-validation gap. Dotted vertical lines mark learning-rate resets caused by SLURM timeout and job resumption; NT-Genus experienced two such resets (epochs 7 and 11) before the resume logic was corrected. Overfitting decreases systematically as data volume increases. 47
- Figure 4.2 Full validation accuracy curves for NT-Genus (genus, left) and NT-Species (species, right) over all completed epochs. Red dotted vertical lines mark epochs where a SLURM job restart incorrectly reset the cosine LR scheduler, causing a temporary accuracy dip. The green shaded region indicates epochs trained after the resume bug fix was applied, during which the LR schedule correctly continues its cosine decay. 52
- Figure 4.3 Genus RC TTA accuracy versus training data volume (log scale). The horizontal axis counts *training reads including duplicates* (the per-epoch sample count); the corresponding *unique*-read counts are smaller (50M→17.8M, 250M→41.9M unique, against a 42.6M source ceiling). Green circles mark our results at three data scales; the red star marks MetaTransformer’s reported genus recall. The blue dashed line is a log-linear fit to our three data points. Per-10× accuracy gains diminish from +7.76 pp (500K→5M) to +4.02 pp (5M→50M). The fourth green circle (v15, 250M) continues the same warm-started series but is held out of the fit; at 67.29% it falls well below the ~71% the log-linear fit would project at that scale—the trajectory flattens off its own projection, directly demonstrating that accuracy saturates beyond 50M rather than continuing along the trend. The MetaTransformer star marks reported genus recall (98.3%); its x-position is based on training throughput (300K steps × batch 2,048) and is not directly comparable to our dataset-size axis. 54
- Figure 4.4 RC TTA benefit across experiments. Top: forward-only vs. RC TTA accuracy (blue: NT-v2 backbone; green: DNABERT family; right of second dashed line: random init). Bottom: absolute gain in percentage points. NT-v2-based models gain +0.77–+1.54 pp consistently. DNABERT gains only +0.58 pp, likely because its overlapping 6-mer tokenization is inherently more RC-symmetric than NT-v2’s non-overlapping 6-mer. DNABERT-2 (BPE tokenization) gains +1.53 pp, similar to early NT-v2 experiments. The randomly initialized Shallow-Genus gains only +0.08 pp, confirming that the benefit primarily corrects pre-training-induced strand biases. 59

Figure 4.5 Confounded backbone comparison: pre-trained NT-v2 + LoRA (NT-Genus, green) vs. one-layer randomly initialized Transformer (Shallow-Genus, purple), both trained on 50M balanced reads. The gaps are real and include depth and width, not pre-training alone. 63



LIST OF TABLES



Table 3.1	Token counts for a 150 bp read under different k -mer tokenization settings.	26
Table 3.2	LoRA hyperparameters.	27
Table 3.3	Comparison of backbone configurations. The one-layer column is the confounded control; the depth-sweep models reuse the same tokenizer, attention-pooling head, and 6-mer vocabulary, and vary only encoder depth.	28
Table 3.4	Training optimization hyperparameters.	32
Table 4.1	Frozen embedding baselines for genus classification (120 classes, 500K dataset).	46
Table 4.2	Architecture and training-technique experiments on the 500K dataset (genus classification). RC TTA results are shown where available. All technique variants after v3 stay within ± 0.5 pp of the v3 RC-TTA plateau except the failed logit-adjustment runs.	47
Table 4.3	Genus classification results across data scales: v4 (500K), NT-Genus-5M (5M balanced), and NT-Genus (50M balanced).	49
Table 4.4	Training dynamics for NT-Genus-5M (5M balanced, 30 epochs). The gap between training and validation accuracy remains small throughout, in sharp contrast to the 500K experiments.	50
Table 4.5	Configuration comparison between NT-Genus-5M and NT-Genus.	50
Table 4.6	Comparison between NT-Genus-5M and NT-Genus (50M) for genus classification (120 classes).	51
Table 4.7	Data scaling analysis: genus classification performance across data volumes. The 250M entry is the warm-started model (initialised from the 50M checkpoint and held out of the log-linear fit); the from-scratch 250M variant instead reaches 64.84%. Only accuracy is reported for the 250M saturation experiment.	52
Table 4.8	Effect of the training balancing axis at a fixed 17.6M-read budget (same reads, balancing differs only in whether species or genera are equalised). Genus-balancing trades a +10 pp gain in macro/flat accuracy for a -23 pp loss in per-read accuracy and a sharp drop in real-sample abundance fidelity.	56
Table 4.9	Effect of RC TTA across experiments. RC TTA consistently improves accuracy at the cost of $2\times$ inference time.	57
Table 4.10	Computational resource usage across experiments.	59
Table 4.11	Resource bottleneck assessment on TWCC H100 nodes.	59
Table 4.12	Comparison between our approach and MetaTransformer [8].	60
Table 4.13	Confounded backbone comparison: NT-v2 (NT-Genus) vs. one-layer Transformer (Shallow-Genus). Tokenizer, head, and 50M reads are shared; depth, width, and pre-training are not.	62
Table 4.14	Confounded backbone results on the original 50M splits: pre-trained NT-v2 (NT-Genus) vs. one-layer Transformer (Shallow-Genus). The -13.19 pp RC-TTA gap is real; it is not an isolated pre-training effect.	62
Table 4.15	Depth sweep, non-overlapping 6-mer, 50M gut reads, RC TTA on the same clean_common 100K file. From-scratch models train 100% of parameters. NT-v2 is LoRA-tuned ($\sim 1\%$ of 500M).	64

Table 4.16	Depth-matched 6-mer comparison at 5M gut reads (RC TTA). The 16-layer from-scratch model is scored on <code>clean_common</code> . NT-v2 v8's 63.05% is its recorded internal-validation figure (that checkpoint was not re-scored on <code>clean_common</code>); models we could check agree across the two pools (8-layer 0.6539 / 65.44%, 16-layer 0.6772 / 67.94%, NT-v2 v9 0.6707 / 67.08%), and the 16-layer 5M run itself scored 0.5411 on both.	65
Table 4.17	Depth-matched pre-training contribution (16-layer from-scratch vs. NT-v2, non-overlapping 6-mer). Pre-training buys data efficiency, not a higher 50M ceiling.	65
Table 4.18	Tokenization ablation (genus-level, 120 classes): NT-v2 + LoRA vs. MetaTransformer under six tokenization configurations. All MetaTransformer models are trained from scratch on 50M balanced reads. RC TTA is unavailable for MetaTransformer.	66
Table 4.19	Tokenization ablation (species-level, 1,535 classes): MetaTransformer under six tokenization configurations, trained on the same 50M balanced reads.	66
Table 4.20	Single-factor changes from a common from-scratch MetaTransformer 6-mer (non-overlapping) baseline at 50M reads (47.00% forward genus Top-1). Overlapping 12-mer and 13-mer hold the MetaTransformer architecture fixed. The NT-v2 row additionally changes depth, width, pre-training, and codebase, and is not an isolated pre-training effect (Section 4.8). All values are forward-only.	68
Table 4.21	Train-fit ceiling at 250M reads: the 6-mer models cannot fit the training set beyond ~66%, whereas an overlapping 13-mer model fits it to ~99%. Training and validation accuracy are essentially equal for the 6-mer models, ruling out overfitting and isolating the input representation as the bottleneck. NT-v2 figures are forward micro-accuracy; the MetaTransformer training figure is micro-precision.	69
Table 4.22	Pre-trained GFM backbone comparison at 5M balanced reads (genus-level, 120 classes). All models use LoRA $r = 16$ fine-tuning and the same attention-pooling head. RC TTA is applied to all models.	70
Table 4.23	Sample-level genus evaluation of the three pre-trained GFM backbones at 5M balanced reads (RC TTA predictions, 5M validation pool, 50K reads/sample).	72
Table 4.24	Same-architecture tokenizer comparison on 50M gut reads, scored on <code>clean_common</code> 100K with RC TTA. The 16-layer non-overlapping 6-mer and 29-layer 6-mer rows are local runs; Jobs A/B/C were trained on Taiwan-2. MetaTransformer 13-mer is the one-block published-protocol number from Table 4.18, not a 16-layer run.	73
Table 4.25	Species Top-1 on the independent 100K-read test set (1,535 classes). Per-genus (50M) is 81 dedicated NT-v2 heads with a frozen backbone. MT hierarchical uses a single genus router and species-logit masking.	76
Table 4.26	Genus sample-level evaluation under one protocol (100 partition samples, 200 sparse samples, 50K reads). Read accuracies are 67.07% (NT-Genus, RC TTA), 48.82% (MT 6-mer $s=1$), and 87.42% (MT 13-mer $s=1$).	80

Table 4.27 Species-level sample evaluation on the independent 100K set (1,535 classes, 1K reads/sample). Kraken 2 uses a custom $k=35$ database from the same 2,505 UMGS+HGR genomes; unclassified reads (30%) count in the denominator but add zero predicted abundance. The MT 13-mer hierarchical species checkpoint did not converge (val_loss = 9.29 at batch 10,000) and is omitted here; its read-level numbers in Table 4.25 remain valid. 80



List of Symbols and Abbreviations



Abbreviations

AMP	Automatic Mixed Precision
ART	Artificial Read Technology (read simulator)
BPE	Byte Pair Encoding
DNA	Deoxyribonucleic Acid
ESM	Evolutionary Scale Modeling
FFN	Feed-Forward Network
GFM	Genomic Foundation Model
GPU	Graphics Processing Unit
HGR	Human Gastrointestinal Reference
LA	Logit Adjustment
LCA	Lowest Common Ancestor
LoRA	Low-Rank Adaptation
LR	Learning Rate
MLM	Masked Language Modeling
MLP	Multi-Layer Perceptron
NT	Nucleotide Transformer
PEFT	Parameter-Efficient Fine-Tuning
RC	Reverse Complement
RC TTA	Reverse-Complement Test-Time Augmentation
RoPE	Rotary Position Embedding
SLURM	Simple Linux Utility for Resource Management
TTA	Test-Time Augmentation
UMGS	Unified Human Gastrointestinal Genome collection
VRAM	Video Random Access Memory

Symbols

r	LoRA rank
α	LoRA scaling factor
K	Number of classes
T	Sequence length (number of tokens)
d	Hidden dimension of the backbone model
M	Number of learnable query tokens in attention pooling
W_Q, W_K, W_V	Query, key, and value projection matrices
A, B	LoRA low-rank decomposition matrices
τ	Temperature parameter for logit adjustment
π_k	Prior probability of class k
f_θ	Classifier parameterized by θ
$\bar{r}$	Reverse complement of read r



Chapter 1

Introduction



1.1 Background and Motivation

Metagenomics, the culture-independent study of microbial communities through direct sequencing of environmental DNA, has transformed our understanding of microbial ecology, human health, and environmental science [1]. Modern metagenomic sequencing platforms, particularly Illumina short-read technologies, generate hundreds of millions of reads per sample, each typically 100–300 base pairs (bp) in length. A fundamental computational challenge in metagenomic analysis is *taxonomic classification*: assigning each sequencing read to the correct taxon in the microbial taxonomy, such as genus or species [2].

Accurate taxonomic classification underpins virtually all downstream metagenomic analyses, including community composition profiling, pathogen detection, antibiotic resistance gene tracking, and comparative microbiome studies [3]. However, the classification task is inherently difficult due to several factors:

- **Short read length:** Individual reads of 150 bp carry limited phylogenetic signal, especially when distinguishing closely related taxa.
- **High taxonomic diversity:** A typical metagenomic sample may contain reads from hundreds to thousands of species spanning dozens of genera.
- **Class imbalance:** Community compositions follow highly skewed abundance distributions, with a few dominant taxa and many rare ones.

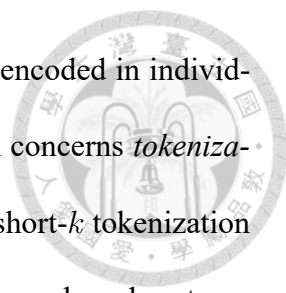
- **Sequence similarity:** Closely related taxa share substantial genomic homology, making fine-grained discrimination (e.g., species-level) particularly challenging.



Traditional approaches to metagenomic classification fall broadly into two categories. *Alignment-based methods* such as BLAST [4] and DIAMOND [5] align reads against reference databases, achieving high precision but at considerable computational cost. *k-mer-based methods* such as Kraken2 [6] and Centrifuge [7] index reference genomes using *k*-mer hash tables, enabling ultra-fast classification at the expense of sensitivity. More recently, deep learning approaches have emerged, including MetaTransformer [8], which applies a Transformer architecture to learned *k*-mer embeddings for metagenomic read classification.

Concurrently, the field of genomics has witnessed the development of *genomic foundation models* (GFMs)—large-scale pre-trained models that learn general-purpose representations of DNA sequences from billions of nucleotides [9]. Models such as the Nucleotide Transformer [9], DNABERT [10], DNABERT-2 [11], and Evo [12] have demonstrated strong performance across diverse genomic tasks, including promoter prediction, splice site identification, and variant effect prediction. These models offer a compelling alternative to task-specific feature engineering: by pre-training on vast genomic corpora, they capture rich sequence semantics that can be transferred to downstream tasks through fine-tuning.

While GFMs have been evaluated extensively on promoter, enhancer, splice-site, TF-binding, and other molecular phenotype prediction tasks, their behavior in high-cardinality metagenomic short-read taxonomic classification remains poorly characterized. A naïve approach—extracting mean-pooled embeddings from a frozen pre-trained model and train-



ing a linear classifier—discards the positional and local information encoded in individual token representations. A separate but equally important question concerns *tokenization*: most existing GFMs (including NT-v2) use non-overlapping or short- k tokenization schemes inherited from masked-language-model pre-training, yet k -mer-based metagenomic classifiers commonly rely on overlapping long k -mers (e.g., $k = 12\text{--}13$) for taxonomic specificity. It is therefore unclear whether large-scale GFM pre-training can compensate for tokenization choices that are mismatched to the downstream taxonomic task.

This thesis investigates two complementary questions: (i) whether a *token-level* approach, preserving the full sequence of token embeddings and employing parameter-efficient fine-tuning, can substantially improve metagenomic taxonomic classification over frozen-embedding baselines; and (ii) whether GFM pre-training can compensate for a short non-overlapping k -mer tokenizer, or whether tokenization imposes a structural ceiling that neither depth nor pre-training can raise. A third, operational question follows from (ii): if pre-training does not raise the 50M-read ceiling, does it still buy data efficiency when labelled reads are scarce?

1.2 Problem Statement

Given a set of metagenomic short reads $\{r_1, r_2, \dots, r_N\}$ generated by sequencing a microbial community, the taxonomic classification problem is to assign each read r_i to its correct taxon $y_i \in \{1, 2, \dots, K\}$, where K is the number of classes at a given taxonomic level (e.g., genus or species).

This thesis formulates the problem as a supervised multi-class classification task. We assume access to a reference genome database from which simulated reads with known taxonomic labels can be generated for training. The challenge is to learn a classifier f_θ :

$\mathcal{R} \rightarrow \{1, \dots, K\}$ that generalizes well from simulated training data to accurately classify novel reads.



The specific objectives of this thesis are:

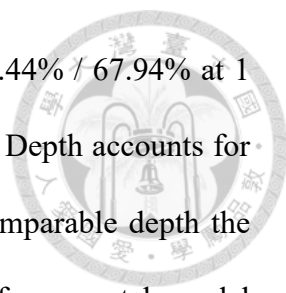
1. **Token-level representation:** Investigate whether preserving the full sequence of token embeddings from a genomic foundation model, rather than using mean-pooled summaries, improves taxonomic classification accuracy.
2. **Parameter-efficient fine-tuning:** Evaluate the effectiveness of LoRA (Low-Rank Adaptation) [13] for adapting a 498M-parameter pre-trained model to the taxonomic classification task with minimal trainable parameters.
3. **Data scaling:** Quantify the relationship between training data volume and classification performance within a fixed GFM-based architecture, and determine the data requirements for achieving competitive accuracy.
4. **Pre-training vs. depth vs. tokenization:** Separate the contributions of large-scale genomic pre-training, encoder depth, and k -mer tokenization, including a depth-matched from-scratch control that the one-layer versus 29-layer comparison does not provide, and identify the dominant bottleneck of the proposed approach.
5. **Multi-level and sample-level evaluation:** Assess performance across multiple metrics (micro accuracy, macro F1, top- k accuracy) at both genus and species taxonomic levels, and characterize how per-read accuracy translates to sample-level utility (relative-abundance estimation, presence/absence detection).

1.3 Contributions

The main contributions of this thesis are as follows:



1. **Systematic evaluation of GFM fine-tuning for metagenomic classification:** We propose a token-level classification pipeline that feeds the complete per-token output of NT-v2 into a learnable attention-pooling head and fine-tunes the backbone with LoRA, covering three GFMs (NT-v2, DNABERT, DNABERT-2) across frozen-embedding baselines, parameter-efficient fine-tuning, data scaling, RC test-time augmentation, and sample-level utility. This is, to our knowledge, the first end-to-end empirical study of GFM fine-tuning for short-read taxonomic classification at the 120-genus and 1,535-species scale.
2. **Data scaling within a fixed GFM-based architecture:** We conduct systematic data-scaling experiments from 500K to 250M reads within the fixed NT-v2 6-mer framework. The data reveal a log-saturation pattern: 500K $\rightarrow$ 5M yields +7.76 pp genus accuracy, 5M $\rightarrow$ 50M adds +4.02 pp, yet a further $5\times$ increase to 250M contributes only +0.22 pp—empirically refuting log-linear scaling projections. This saturation is tokenization-bound rather than data-bound: an overlapping 13-mer model trained on the same data range continues scaling from 87.5% to 98.7%, confirming that NT-v2’s 6-mer tokenizer is the binding constraint.
3. **Pre-training buys data efficiency, not a higher 50M ceiling:** A one-layer randomly initialized Transformer under the same 6-mer tokenizer, 50M reads, and head trails 29-layer NT-v2 by +13.19 pp RC TTA (67.07% vs. 53.88%). That comparison confounds pre-training with depth and width. A depth sweep with everything else held fixed (NT-v2 6-mer tokenizer, 50M gut reads, attention-pooling head, RC



TTA, the same `clean_common` 100K file) scores 53.92% / 65.44% / 67.94% at 1 / 8 / 16 from-scratch layers, against 67.08% for NT-v2 LoRA. Depth accounts for +14.02 pp (1 $\rightarrow$ 16); at matched 6-mer tokenization and comparable depth the net contribution of pre-training at 50M is $\approx$ 0 (the 16-layer from-scratch model is +0.86 pp higher, with $129\times$ fewer parameters). The same 16-layer comparison at 5M reads reverses the ranking: NT-v2 63.05% versus from-scratch 54.11% (+8.94 pp). Pre-training therefore helps when labels are scarce; it does not raise the 6-mer ceiling once 50M labelled reads are available. The 5M NT-v2 figure is the recorded internal-validation number for that checkpoint; every model we could re-score agrees across the two pools (Section 4.8).

4. Tokenization is the dominant factor, including at matched architecture: Holding MetaTransformer’s architecture fixed and varying $k \in \{6, 12, 13\}$ and stride exposes two compounding effects: overlapping stride ($s=1$) vs. non-overlapping ($s=k$) accounts for up to +60 pp at $k=13$, and longer k monotonically improves accuracy among overlapping configurations. A MetaTransformer with overlapping 13-mer tokenization trained from scratch surpasses NT-v2 + LoRA by +20.35 pp (87.42% vs. 67.07%). A same-architecture 16-layer comparison on `clean_common` RC TTA reaches 83.83% with a hashed 13-mer (2 GiB) and 85.63% with an exact 13-mer, versus 67.94% for the 16-layer non-overlapping 6-mer; a 29-layer from-scratch 6-mer reaches only 69.52%. Depth can compensate for a weak tokenizer; it does not break the 6-mer ceiling. The 6-mer constraint—not backbone capacity, LoRA, or pre-training—is the principal bottleneck.

5. Reverse-complement test-time augmentation: We evaluate RC TTA—averaging logits from forward and reverse-complement strand passes at inference—across

all experimental settings and architectures. RC TTA delivers a consistent, zero-training-cost improvement of +0.1 to +1.5 pp, with a clear differential between pre-trained models (+0.78–+1.54 pp) and randomly initialized models ($\approx +0.08$ pp), implicating pre-training-induced strand bias as the primary mechanism.

6. **Hierarchical species classification and sample-level utility:** We extend the evaluation framework to species-level classification (1,535 classes) via a hierarchical genus-routing pipeline, and to sample-level abundance estimation. Per-genus LoRA specialists under oracle-genus routing reach 29.5% species Top-1, exceeding the 27.4% flat-classifier oracle and validating hierarchical specialisation; the genus-router accuracy (66%–67%) is identified as the binding bottleneck for end-to-end routing. At the sample level, 67% per-read accuracy on species-balanced synthetic samples translates to Pearson $r = 0.993$ for relative abundance and 100% detection sensitivity at $\geq 1\%$ abundance, though generalization to real and compositionally variable communities remains future work.

1.4 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** reviews the background and related work, covering metagenomic classification methods, genomic foundation models, parameter-efficient fine-tuning, and attention mechanisms.
- **Chapter 3** presents the proposed methodology in detail, including the model architecture, training strategy, data pipeline, and evaluation framework.
- **Chapter 4** describes the experimental setup and presents comprehensive results,

including ablation studies, data scaling analysis, species-level evaluation, and computational resource assessment.

- **Chapter 5** summarizes the findings, discusses limitations, and outlines directions for future work.

Chapter 2

Background and Related Work



This chapter provides the necessary background for understanding the methods proposed in this thesis. We review existing approaches to metagenomic taxonomic classification, introduce genomic foundation models, describe parameter-efficient fine-tuning techniques, and discuss attention-based sequence aggregation mechanisms.

2.1 Metagenomic Taxonomic Classification

Metagenomic taxonomic classification aims to assign each sequencing read from an environmental sample to its source organism’s position in the phylogenetic tree. We review three major paradigms for this task.

2.1.1 Alignment-Based Methods

Alignment-based methods classify reads by aligning them to reference genome databases. BLAST [4] performs local sequence alignment using heuristic seed-and-extend strategies, achieving high sensitivity but at significant computational cost (hours to days for typical metagenomic datasets). DIAMOND [5] accelerates protein-level alignments through double indexing and optimized scoring, achieving up to $20,000\times$ speedup over BLAST while maintaining comparable sensitivity. MegaBLAST [14] optimizes nucleotide-level alignment for closely related sequences.

The primary limitation of alignment-based methods is their computational cost, which scales poorly with the size of modern metagenomic datasets containing hundreds of mil-

lions of reads.



2.1.2 k -mer-Based Methods

k -mer-based methods represent sequences as sets or distributions of fixed-length sub-sequences (k -mers) and use hash-table lookups for rapid classification.

Kraken2 [6] constructs a database mapping each k -mer ($k = 35$) to its lowest common ancestor (LCA) in the taxonomic tree. Classification proceeds by querying each read's k -mers against this database and assigning the read to the taxon that covers the most k -mers. Kraken2 achieves classification speeds exceeding 10^6 reads per minute, making it one of the most widely adopted tools in metagenomic analysis.

Centrifuge [7] uses a compressed FM-index to perform rapid exact matching of reads against reference genomes, achieving lower memory requirements than Kraken2 while maintaining competitive accuracy.

While k -mer methods excel in speed, they rely on exact k -mer matches and thus struggle with sequences that diverge from the reference database. They also lack the ability to capture higher-order sequence patterns beyond individual k -mers.

2.1.3 Deep Learning Methods

Deep learning methods learn discriminative representations directly from sequence data, potentially capturing complex patterns that elude k -mer matching.

DeepMicrobes [15] applies a bidirectional LSTM with attention to one-hot encoded nucleotide sequences, achieving competitive performance on species-level classification

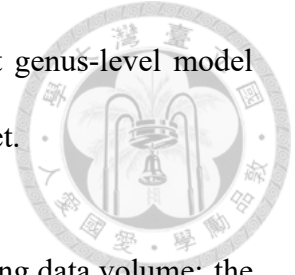
but requiring substantial training data and compute.

BERTax [16] adapts the BERT architecture to DNA taxonomic classification at superkingdom, phylum, and genus levels. Rather than relying on sequence similarity to a reference database, BERTax learns contextual k -mer representations and demonstrates generalisation to novel organisms with no close relatives in the training set. This work establishes the viability of transformer-based language models for read-level taxonomic assignment, motivating the use of larger pre-trained GFM in this thesis.

MetaTransformer [8] is the most directly relevant prior work to this thesis. It employs a custom Transformer architecture that operates on learned 12-mer embeddings. Key design choices include:

- **12-mer tokenization:** Input reads are segmented into non-overlapping 12-mers, each mapped to a learnable embedding vector. The choice of $k = 12$ balances vocabulary size ($4^{12} \approx 16.7\text{M}$ possible 12-mers, though only $\sim 1.4\text{M}$ appear in training data) against sequence granularity.
- **Transformer encoder:** A Transformer encoder (the best-performing configuration uses $d_{\text{model}} = 64$ with a single encoder block for $k = 13$; the authors also evaluate up to 4 blocks and $d_{\text{model}} = 128$) processes the sequence of k -mer embeddings.
- **Mean pooling:** Token representations are mean-pooled into a fixed-length vector for classification.
- **Large-scale training:** The model is trained on balanced reads simulated from 2,505 HGR-UMGS genomes using the ART Illumina simulator with a coverage factor of 3 (the exact total number of reads is not reported in the paper; we estimate $\sim 614\text{M}$

based on 300K training steps $\times$ batch size 2,048). The best genus-level model achieves 98.9% precision and 98.3% recall on the validation set.



A key insight from MetaTransformer is the critical role of training data volume: the authors demonstrate that performance scales substantially with data size, and that balanced sampling across taxa is essential for achieving high macro-level performance.

2.2 Genomic Foundation Models

Genomic foundation models (GFMs) are large-scale neural networks pre-trained on vast corpora of DNA sequences using self-supervised learning objectives. Inspired by the success of language models such as BERT [17] and GPT [18], GFMs treat DNA sequences as a “language” of nucleotides and learn contextual representations that capture biological structure and function.

2.2.1 Nucleotide Transformer

The Nucleotide Transformer (NT) [9] family of models, developed by InstaDeep, are among the largest and most widely evaluated GFMs. This thesis uses the `nucleotide-transformer-v2-500m-multi-species` variant, which features:

- **Architecture:** An ESM-2-based [19] Transformer encoder with 29 layers, 1,024 hidden dimensions, and 16 attention heads.
- **Tokenization:** A 6-mer tokenizer that encodes DNA sequences into non-overlapping 6-nucleotide tokens. This produces a vocabulary of $4^6 = 4,096$ possible tokens plus special tokens.

- **Pre-training:** Masked language modeling (MLM) on 850 genomes from 174 species, spanning ~ 3.2 billion nucleotides.
- **Total parameters:** 498M parameters.



The model produces contextual embeddings for each token in the input sequence, yielding a representation tensor of shape $[\text{batch}, \text{seq_len}, 1024]$ that captures both local sequence composition and long-range dependencies.

2.2.2 Other Genomic Foundation Models

DNABERT [10] was one of the first BERT-style models pre-trained on human genome sequences using overlapping k -mer tokenization ($k = 3, 4, 5, 6$). While effective for tasks such as promoter prediction, its relatively small scale (110M parameters) and human-genome-only pre-training limit its applicability to diverse metagenomic sequences.

DNABERT-2 [11] replaces k -mer tokenization with Byte Pair Encoding (BPE), eliminating the need to choose a fixed k and enabling more flexible sequence representation. It also adopts a multi-species pre-training strategy.

HyenaDNA [20] adopts single-nucleotide tokenization combined with sub-quadratic Hyena convolutions, enabling context windows up to 1M tokens at a fraction of the compute cost of attention-based models. Its single-character vocabulary eliminates the k design choice entirely, representing the opposite end of the tokenization design space from k -mer models.

Caduceus [21] is the first RC-equivariant bidirectional DNA language model. By constructing BiMamba layers with hard-coded reverse-complement weight sharing, Ca-

duceus enforces strand symmetry at the architectural level rather than relying on augmentation or post-hoc inference tricks. This is directly relevant to the RC consistency analysis in Section 2.5.1; unlike Caduceus, the NT-v2 backbone does not natively enforce RC equivariance, motivating our use of RC training augmentation and RC TTA.

Evo [12] is a 7B-parameter model based on the StripedHyena architecture [22], capable of modeling DNA sequences up to 131K nucleotides. Its successor, **Evo 2** [23], scales this approach to 7B and 40B parameters trained on 9.3 trillion bases spanning all domains of life, with a context window of up to 1M tokens at single-nucleotide resolution. We adopt the 7B Evo 2 variant as a frozen-embedding baseline (Section 4.2). While powerful for long-range genomic modeling, the large size of both models makes them challenging to deploy for metagenomic classification tasks involving hundreds of millions of short reads.

Across these models, a consistent theme emerges: tokenization strategy is a critical design axis. Lindsey, Pershing, Habib, *et al.* [24] systematically evaluate character, BPE, k -mer, Unigram, and WordPiece tokenizers across 44 genomic classification tasks and find accuracy differences of up to 5 percentage points depending on tokenizer choice, corroborating our own ablation results in Chapter 4.

2.2.3 GFMs for Metagenomic Classification

GFMs have demonstrated strong performance on a range of molecular phenotype benchmarks, yet their systematic evaluation under high-cardinality multi-class taxonomy—spanning hundreds of genera or thousands of species from short metagenomic reads—remains absent from the literature. A naïve approach extracts mean-pooled embeddings

from a frozen pre-trained model and trains a downstream classifier (e.g., MLP or SVM) on these fixed representations. While simple, this approach discards the token-level structure of the representation.

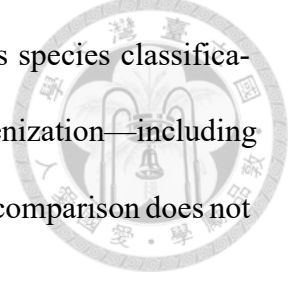


Several recent works have begun to close this gap, though none performs systematic benchmarking across tokenization strategies at the scale of 120-genus or 1,535-species classification. **ViraLM** [25] fine-tunes DNABERT-2 for binary virus/non-virus classification of metagenomic contigs, reporting a 22% F1 improvement on short fragments — demonstrating that GFM fine-tuning is beneficial even on compositionally diverse metagenomic sequences. **DNABERT-S** [26] adapts DNABERT-2 with contrastive learning to produce species-discriminative embeddings, achieving improvements on metagenomic binning and species clustering tasks; however, it operates at the embedding level rather than performing direct read classification. Both works fine-tune pre-existing GFMs and treat metagenomics as a secondary application; neither performs systematic tokenization ablation nor evaluates at the genus-level multi-class classification scale (120 genera, 50M reads) addressed in this thesis.

Two GFMs pre-trained on metagenomic sequence have since been released and are in-domain relative to this task in a way NT-v2 is not. **METAGENE-1** [27] is a 7B-parameter autoregressive model trained on ~ 1.5 trillion base pairs of wastewater metagenomes with BPE tokenization. **GenomeOcean** [28] is a 4B-parameter model trained on ~ 600 Gbp of metagenomic assemblies, also with BPE. We do not evaluate either model; Chapter 5 states this as a scope limitation. Their BPE tokenizers would additionally confound a pre-training-only comparison with NT-v2.

This thesis addresses the remaining gap by systematically fine-tuning and bench-

marking *general-purpose* GFM for 120-class genus and 1,535-class species classification, and by separating data scale, encoder depth, pre-training, and tokenization—including a depth-matched from-scratch control that a one-layer versus 29-layer comparison does not provide.



2.3 Parameter-Efficient Fine-Tuning

Fine-tuning the full parameter set of a 498M-parameter model requires substantial compute and memory, and risks catastrophic forgetting of the pre-trained representations. Parameter-efficient fine-tuning (PEFT) methods address this by adapting only a small subset of parameters while keeping the majority frozen.

2.3.1 Low-Rank Adaptation (LoRA)

LoRA [13] is based on the hypothesis that weight updates during fine-tuning lie in a low-rank subspace. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA parameterizes the update as:

$$W = W_0 + \Delta W = W_0 + BA \quad (2.1)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $\text{rank } r \ll \min(d, k)$. During training, only A and B are updated while W_0 remains frozen. The forward pass becomes:

$$h = W_0 x + \frac{\alpha}{r} B A x \quad (2.2)$$

where α is a scaling hyperparameter that controls the magnitude of the adaptation.

Key advantages of LoRA include:

- **Parameter efficiency:** With $r = 16$ applied to query, key, and value projections across 29 Transformer layers, only 5.54M parameters (1.11% of total) are trainable.
- **No additional inference latency:** The adapted weights $W_0 + BA$ can be merged, adding no overhead at inference time.
- **Modular:** Different LoRA adapters can be trained for different tasks and swapped at deployment time.

2.3.2 Other PEFT Methods

Adapter layers [29] insert small trainable bottleneck modules between Transformer layers. While effective, they introduce additional inference latency due to the extra forward-pass computation.

Prefix tuning [30] prepends learnable “virtual tokens” to the input sequence, influencing the attention computation without modifying model weights. It is memory-efficient but can struggle with tasks requiring fine-grained output control.

We choose LoRA for this thesis due to its combination of strong performance, parameter efficiency, and the absence of additional inference latency.

2.4 Attention Mechanisms for Sequence Aggregation

Given a sequence of token embeddings $\{h_1, h_2, \dots, h_T\} \in \mathbb{R}^{T \times d}$, a classification task requires aggregating these into a fixed-dimensional representation. The choice of aggregation mechanism directly impacts what information is preserved.



2.4.1 Mean Pooling

The simplest approach averages all token embeddings:

$$z = \frac{1}{T} \sum_{t=1}^T h_t \quad (2.3)$$

Mean pooling is computationally trivial but weights all tokens equally, potentially diluting the contribution of discriminative tokens with non-informative ones.

2.4.2 Attention Pooling

Attention pooling uses learnable query vectors to compute a weighted combination of token embeddings. Given M learnable query vectors $\{q_1, \dots, q_M\} \in \mathbb{R}^{M \times d_k}$ and linear projections $W_K, W_V \in \mathbb{R}^{d \times d_k}$:

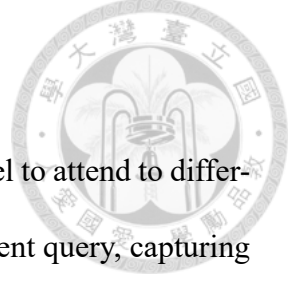
$$K = HW_K, \quad V = HW_V \quad (2.4)$$

$$\alpha_{m,t} = \frac{\exp(q_m \cdot k_t / \sqrt{d_k})}{\sum_{t'} \exp(q_m \cdot k_{t'} / \sqrt{d_k})} \quad (2.5)$$

$$z_m = \sum_{t=1}^T \alpha_{m,t} v_t \quad (2.6)$$

$$z = \text{Concat}(z_1, \dots, z_M) \in \mathbb{R}^{M \cdot d_k} \quad (2.7)$$

This mechanism enables the model to learn which positions in the DNA sequence are most informative for classification, automatically focusing on discriminative k -mers while down-weighting uninformative regions.



2.4.3 Multi-Head Attention Pooling

Extending attention pooling with multiple heads allows the model to attend to different aspects of the sequence simultaneously. Each head learns a different query, capturing diverse patterns (e.g., one head may focus on conserved marker genes while another attends to variable regions). The concatenated outputs from all heads provide a rich, multi-faceted representation for downstream classification.

2.5 Data Augmentation for DNA Sequences

2.5.1 Reverse-Complement Symmetry in DNA

DNA is a double-stranded molecule: the two antiparallel strands are linked by Watson–Crick base pairing ($A \leftrightarrow T$, $C \leftrightarrow G$). Given a sequence $s = s_1 s_2 \cdots s_L$ on the forward ($5' \rightarrow 3'$) strand, its *reverse complement* $\bar{s} = \overline{s_L} \cdots \overline{s_2 s_1}$ on the complementary strand encodes identical biological information: both correspond to the same genomic locus and therefore the same organism.

In shotgun metagenomic sequencing, a read is equally likely to originate from either strand. Consequently, a biologically correct classifier must satisfy the *strand symmetry* property:

$$f(s) = f(\bar{s}) \quad \forall s \quad (2.8)$$

However, standard neural network architectures do not inherently enforce this constraint. Zhou et al. [31] systematically demonstrate that deep learning models for genomics frequently violate strand symmetry, producing inconsistent predictions for a sequence and its reverse complement. Ma [32] further show that even DNA language models fine-tuned with RC data augmentation still exhibit substantial prediction inconsistency.

2.5.2 Strategies for Exploiting RC Symmetry



Three complementary strategies have been proposed to address the strand symmetry problem:

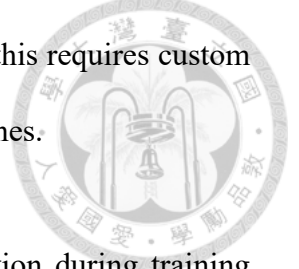
1. **Random RC training augmentation:** During training, each read is replaced with its reverse complement with probability $p = 0.5$, exposing the model to both strand orientations. While this encourages—but does not guarantee—strand-invariant representations, it effectively doubles the training data diversity at zero storage cost.
2. **RC test-time augmentation (RC TTA):** At inference, both the forward read s and its reverse complement $\bar{s}$ are processed independently, and their output logits are averaged before taking the argmax prediction:

$$\hat{y} = \arg \max_k \frac{1}{2} [f_{\theta}(s) + f_{\theta}(\bar{s})] \quad (2.9)$$

This “post-hoc conjoined” approach [31] enforces strand symmetry at inference time without modifying the trained model. It can be understood as a two-view ensemble where both views observe the same biological entity from different strand orientations. By the standard ensemble argument, averaging two correlated but noisy estimates of the same ground-truth label reduces prediction variance and cancels orientation-dependent errors. Zhou et al. [31] find that this approach consistently performs well across diverse genomic tasks.

3. **RC consistency training:** A regularization loss (e.g., KL divergence) explicitly penalizes divergence between $f_{\theta}(s)$ and $f_{\theta}(\bar{s})$ during training, producing a single model that is intrinsically more strand-invariant without requiring double inference at test time [32]. An alternative is to design *RC-equivariant architectures* that hard-

code strand symmetry into the network structure [33], though this requires custom layers incompatible with pre-trained foundation model backbones.



In this thesis, we adopt strategies (1) and (2): RC augmentation during training and RC TTA during evaluation. We also experiment with RC consistency training (Section 4.3.1) but find that its interaction with other regularization techniques (e.g., logit adjustment) can be detrimental, and RC TTA alone provides a reliable +1–1.5 percentage point improvement across all experimental settings (Section 4.5).

2.6 Handling Class Imbalance

Metagenomic samples exhibit severe class imbalance: a few dominant taxa may constitute the majority of reads while rare taxa contribute only a handful. This imbalance can bias classifiers toward frequent classes, resulting in high micro accuracy but poor macro F1 due to frequent misclassification of rare taxa.

2.6.1 Loss-Level Approaches

Class-weighted cross-entropy scales the loss for each class inversely proportional to its frequency:

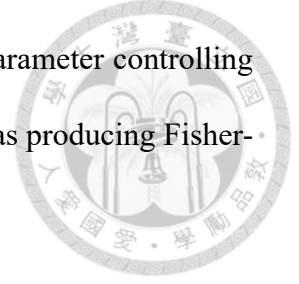
$$\mathcal{L} = - \sum_{k=1}^K w_k y_k \log \hat{y}_k, \quad w_k \propto 1/n_k \quad (2.10)$$

However, this approach can be unstable with mixed-precision training and large class counts [34].

Logit adjustment [35] modifies the output logits by a class-prior-dependent offset:

$$\tilde{f}_k(x) = f_k(x) + \tau \log \pi_k \quad (2.11)$$

where π_k is the prior probability of class k and τ is a temperature parameter controlling the strength of adjustment. This approach is theoretically motivated as producing Fisher-consistent estimators under class imbalance.



2.6.2 Sampling-Level Approaches

Balanced sampling constructs each training batch by sampling classes uniformly and then sampling instances within each class. This ensures that rare classes are seen as frequently as common ones during training.

Class-aware sampling (e.g., `WeightedRandomSampler`) assigns sampling weights inversely proportional to class frequency, achieving a similar effect to balanced batches within standard data loading pipelines.

2.7 Summary

This chapter has reviewed the key concepts underpinning our proposed approach: metagenomic classification methods from alignment-based to deep learning, genomic foundation models that provide rich pre-trained representations, parameter-efficient fine-tuning via LoRA, attention-based sequence aggregation, and strategies for data augmentation and class imbalance. In the next chapter, we describe how these components are integrated into a cohesive classification framework.

Chapter 3

Methodology



This chapter describes the proposed token-level genomic foundation model classification framework. We present the overall system architecture, detail each component, and describe the training strategy, data pipeline, and evaluation methodology.

3.1 System Overview

Figure 3.1 illustrates the end-to-end pipeline. Given a raw DNA read of 150 bp, the system performs the following steps:

1. **Tokenization:** The read is segmented into non-overlapping 6-mer tokens using the Nucleotide Transformer v2 tokenizer.
2. **Token-level encoding:** The token sequence is processed by the pre-trained Nucleotide Transformer v2 backbone (with LoRA adapters), producing contextual embeddings for each token.
3. **Attention pooling:** A multi-head attention pooling head aggregates the token embeddings into a fixed-dimensional classification vector.
4. **Classification:** A feed-forward network maps the pooled representation to class logits.

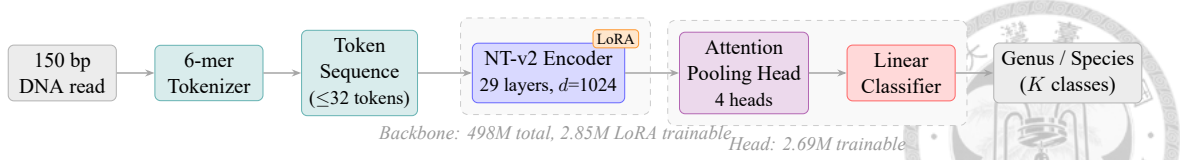


Figure 3.1: Overview of the proposed token-level GFM classification pipeline. A 150 bp DNA read is tokenized into non-overlapping 6-mers, encoded by the LoRA-adapted NT-v2 backbone (29 layers, $d=1024$), aggregated via multi-head attention pooling, and classified into one of K taxonomic classes (120 genera or 1,535 species). Orange badge indicates LoRA adapter modules; dashed boxes delineate the backbone and classification head parameter groups.

3.2 Backbone: Nucleotide Transformer v2

We employ the nucleotide-transformer-v2-500m-multi-species model as the backbone encoder. This model is based on the ESM-2 architecture [19], a protein language model architecture adapted for nucleotide sequences. The selection of NT-v2 over other genomic foundation models is motivated by four considerations:

1. **Empirical performance on metagenomic reads:** A direct comparison of frozen-backbone baselines (Section 4.2) shows that NT-v2 achieves 51.70% genus-level accuracy, outperforming the 7B-parameter Evo-2 model [23] (46.95%) despite having $14\times$ fewer parameters. This result provides direct empirical evidence that NT-v2’s representations are better suited to metagenomic read classification.
2. **Multi-species pre-training:** NT-v2’s multi-species variant is pre-trained on 850 genomes spanning 174 species [9], making its pre-training distribution the closest among available GFMs to the multi-organism nature of metagenomic data. By contrast, DNABERT [10] is pre-trained exclusively on the human genome, which is taxonomically far from gut microbial sequences.
3. **k -mer tokenization as a common ground with MetaTransformer:** MetaTransformer [8] demonstrates that k -mer-based sequence discretization is effective for

metagenomic classification. NT-v2 shares this philosophy through its non-overlapping 6-mer tokenizer, establishing a direct methodological link. This allows the pre-trained 6-mer embeddings to serve as a principled initialization relative to Meta-Transformer’s learned-from-scratch 12-mer embeddings. DNABERT-2 [11], while multi-species, replaces k -mer tokenization with BPE, breaking this alignment.

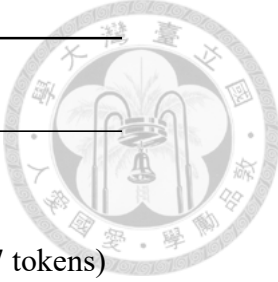
4. **Practical scalability:** At 498M parameters, NT-v2 can be fine-tuned with LoRA on a single H100 GPU with well under 10 GB of VRAM, making it feasible to run hundreds of training epochs over tens of millions of reads. Evo’s 7B parameters [12], while powerful for long-range genomic modeling, impose substantially higher memory requirements that are difficult to justify for 150 bp short-read classification.

3.2.1 Tokenization

Stride and overlap. A k -mer tokenizer slides a window of length k along a DNA sequence, converting each window into a vocabulary token. The *stride* s controls the step size between consecutive windows:

- **Non-overlapping** ($s = k$): each nucleotide belongs to exactly one token. A 150 bp read yields $\lfloor 150/k \rfloor$ tokens.
- **Overlapping** ($s = 1$): every nucleotide position starts a new token. A 150 bp read yields $150 - k + 1$ tokens.

For example, a 12-nucleotide sequence AGTCGATTCGCA tokenized with $k=6$ gives:



Stride	Tokens
$s=6$ (non-overlapping)	AGTCGA TTCGCA (2 tokens)
$s=1$ (overlapping)	AGTCGA, GTCGAT, TCGATC, ... (7 tokens)

Table 3.1 summarises the resulting token counts for a 150 bp read under the configurations used in this thesis.

Table 3.1: Token counts for a 150 bp read under different k -mer tokenization settings.

k	Stride s	Overlap	Tokens/read
6	6	Non-overlapping	25
6	1	Overlapping	145
12	12	Non-overlapping	12
12	1	Overlapping	139

NT-v2 tokenization. The NT-v2 tokenizer uses non-overlapping 6-mers ($k=6$, $s=6$), the setting used during its pre-training. For a 150 bp read this produces 25 tokens (plus [CLS] and [EOS] special tokens). The maximum sequence length is set to 128 tokens, accommodating reads up to ~ 768 bp with padding. Section 4.9 presents a controlled ablation that quantifies the effect of different tokenization settings on classification accuracy.

3.2.2 Encoder Architecture

The backbone consists of 29 Transformer encoder layers, each with:

- Multi-head self-attention with 16 heads and hidden dimension 1,024.
- A feed-forward network (FFN) with intermediate dimension 4,096.
- Pre-layer normalization (LayerNorm before attention and FFN).

- Rotary position embeddings (RoPE) [36] for encoding positional information.

The backbone outputs a tensor $H \in \mathbb{R}^{B \times T \times 1024}$, where B is the batch size and T is the sequence length. We use the *full token sequence* (excluding special tokens) as input to the classification head, rather than extracting only the [CLS] token or applying mean pooling.

3.2.3 LoRA Adaptation

We apply LoRA [13] to the query (W_Q), key (W_K), and value (W_V) projection matrices in each of the 29 self-attention layers. The LoRA configuration is:

Table 3.2: LoRA hyperparameters.

Parameter	Value
Rank r	16
Scaling factor α	32
Dropout	0.05
Target modules	W_Q, W_K, W_V
Trainable LoRA params	2,850,816
Classification head params	2,688,632
Total trainable params	5,539,448 (1.11%)
Total model params	498,623,561

Gradient checkpointing is enabled for the backbone to reduce peak GPU memory usage at the cost of recomputing activations during the backward pass.

3.2.4 Ablation: Shallow Transformer Backbone

To test whether the pre-trained 29-layer NT-v2 backbone is necessary under 6-mer tokenization, we first replace it with a shallow Transformer encoder trained entirely from scratch. This ablation mirrors the encoder depth of MetaTransformer [8] while keeping

the tokenizer fixed to NT-v2 6-mers. It is *not* a depth-matched pre-training control: the comparison confounds pre-training with encoder depth (29 vs. 1) and width ($d=1024$ vs. $d=128$). Chapter 4 therefore reports this one-layer variant as a confounded baseline, then a depth sweep that varies only the number of encoder layers.

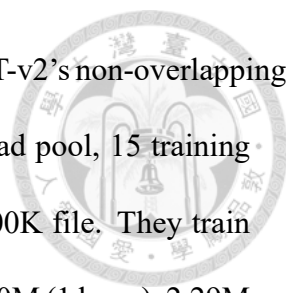
The one-layer Transformer backbone consists of:

- **Learned token embeddings:** A randomly initialized embedding matrix $E \in \mathbb{R}^{V \times d_{\text{model}}}$, where $V = 4,107$ is the NT-v2 6-mer vocabulary size and $d_{\text{model}} = 128$.
- **Learned positional embeddings:** A position embedding matrix $P \in \mathbb{R}^{L_{\text{max}} \times d_{\text{model}}}$, added to the token embeddings.
- **Single-layer Transformer encoder:** One pre-norm Transformer encoder layer with 2 attention heads, feed-forward dimension $d_{\text{ff}} = 512$, GELU activation, and dropout $p = 0.1$.

Table 3.3: Comparison of backbone configurations. The one-layer column is the confounded control; the depth-sweep models reuse the same tokenizer, attention-pooling head, and 6-mer vocabulary, and vary only encoder depth.

Aspect	NT-v2 + LoRA	Shallow (1-layer)
Tokenizer	6-mer (shared)	6-mer (shared)
Encoder layers	29	1
Hidden dimension	1,024	128
Attention heads	16	2
Feed-forward dim	4,096	512
Pre-training	MLM on 850 genomes	None (from scratch)
Backbone params	498M (5.5M trainable)	~200K (all trainable)

The same attention pooling classification head is used for both variants, with the input dimension adjusted to match the backbone output ($d = 1,024$ for NT-v2, $d = 128$ for the shallow variant).



The depth-sweep models (1, 8, 16, and 29 encoder layers) keep NT-v2’s non-overlapping 6-mer tokenizer, the attention-pooling head, the 50M (or 5M) gut-read pool, 15 training epochs, and RC TTA, and are scored on the same `clean_common` 100K file. They train 100% of parameters from scratch (no LoRA). Parameter counts are 0.90M (1 layer), 2.29M (8 layers), 3.88M (16 layers), and 6.45M (29 layers). Because NT-v2 was pre-trained with a 2,048-token context ($\sim 12,288$ bp) and is here fed 25 tokens (150 bp), any transfer is also outside its pre-training length regime [9].

3.2.5 Same-Architecture Tokenizer Variants

A complementary control holds the 16-layer from-scratch encoder fixed and varies only the tokenizer on the same 50M gut reads: non-overlapping 6-mer (25 tokens/read), overlapping 6-mer (146 tokens/read), hashed overlapping 13-mer (139 tokens/read, 2 GiB embedding), and exact overlapping 13-mer (139 tokens/read, 8.6 GiB embedding). These runs are scored on the same `clean_common` 100K file with RC TTA (Section 4.9). The hashed and exact 13-mer arms additionally differ in embedding width and numeric precision, which Chapter 4 discloses rather than treating as a perfectly isolated k ablation.

3.3 Classification Head: Attention Pooling

The classification head aggregates the variable-length token sequence into a fixed-dimensional representation and maps it to class logits. We use a multi-head attention pooling mechanism inspired by Perceiver [37] and Set Transformer [38].



3.3.1 Architecture

The attention pooling head consists of:

1. **Learnable queries:** $h = 4$ attention heads, each with head dimension $d_k = d/h = 256$, where $d = 1024$ is the backbone hidden size. A single learnable query vector per head is stored as $Q \in \mathbb{R}^{h \times d_k}$.
2. **Cross-attention:** The backbone token embeddings $H \in \mathbb{R}^{T \times d}$ are projected to keys and values with $W_K, W_V \in \mathbb{R}^{d \times d}$ (no down-projection) and split across heads. Each head i performs scaled dot-product attention between its query and the per-head key/value slices $K_i, V_i \in \mathbb{R}^{T \times d_k}$:

$$K = HW_K, \quad V = HW_V \quad (3.1)$$

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i, \quad i = 1, \dots, h \quad (3.2)$$

3. **Aggregation:** The h single-vector head outputs are concatenated, yielding a representation of dimension $h \cdot d_k = d = 1024$.
4. **Feed-forward classifier:**

$$\text{LayerNorm}(d) \rightarrow \text{Linear}(d, 512) \rightarrow \text{GELU} \rightarrow \text{Dropout}(p) \rightarrow \text{Linear}(512, K) \quad (3.3)$$

where K is the number of classes.

With $d = 1024$, $h = 4$, and $K = 120$ genera, the head comprises the query (1,024), the key/value projections W_K, W_V with their biases ($2 \times (1,048,576 + 1,024)$), the LayerNorm (2,048), and the two classifier layers ($524,800 + 61,560$), totalling 2,688,632 train-

able parameters (Table 3.2); the species head ($K = 1,535$) correspondingly comprises 3,414,527.



The attention pooling mechanism enables the model to selectively focus on the most taxonomically informative tokens in the sequence, analogous to how a biologist might focus on specific conserved marker regions within a read to determine its taxonomic origin.

3.3.2 Alternative Heads

We also implemented two alternative classification heads for comparison:

Mean Pool Head: Averages all token embeddings along the sequence dimension, followed by the same feed-forward classifier. This serves as the baseline aggregation strategy.

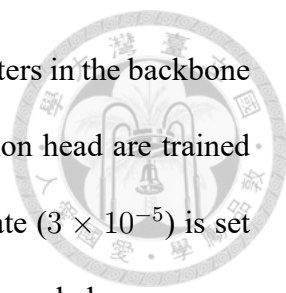
Transformer Pool Head: Applies a single Transformer encoder layer on top of the backbone output before mean pooling, adding another level of self-attention. This explores whether an additional attention layer can compensate for simple mean pooling.

3.4 Training Strategy

3.4.1 Two-Phase Training

We employ a two-phase training strategy to stabilize learning:

Phase 1 — Head-only warmup (5 epochs): The backbone is completely frozen, and only the classification head parameters are trained. This phase initializes the head weights to produce meaningful gradients before the backbone is unfrozen, preventing early gradient noise from corrupting pre-trained representations.



Phase 2 — Joint fine-tuning (up to 25–30 epochs): LoRA adapters in the backbone are unfrozen, and both the backbone (via LoRA) and the classification head are trained jointly. Differential learning rates are used: the backbone learning rate (3×10^{-5}) is set lower than the head learning rate (5×10^{-4}) to preserve pre-trained knowledge.

3.4.2 Optimization

The training procedure uses the following optimization configuration:

Table 3.4: Training optimization hyperparameters.

Parameter	Value
Optimizer	AdamW
Backbone learning rate	3×10^{-5}
Head learning rate	5×10^{-4}
Weight decay	0.02
LR scheduler	Cosine with linear warmup
Warmup ratio	10% of total steps
Gradient accumulation steps	1–2
Mixed precision	bfloat16

Mixed precision training: We use bfloat16 automatic mixed precision (AMP) on NVIDIA H100 GPUs. The bfloat16 format provides the dynamic range of float32 with reduced precision, avoiding the gradient underflow issues that arise with float16 and class-weighted losses.

Early stopping: Training is halted when validation accuracy does not improve for 7 consecutive epochs, and the model with the highest validation accuracy is saved as the final checkpoint.

3.4.3 Transfer Learning Across Experiments

To facilitate progressive scaling from smaller to larger datasets, we implement a transfer learning mechanism (`--init_from`) distinct from the training resumption mechanism (`--resume`).

Resume (`--resume`): Loads the full training state (model weights, optimizer state, epoch counter, learning rate scheduler) from a checkpoint, enabling continuation of the *same* experiment after a job interruption (e.g., SLURM time limit).

Transfer learning (`--init_from`): Loads only the model weights from a checkpoint for a *new* experiment, resetting the epoch counter and optimizer state. Supports partial loading, automatically skipping parameters with shape mismatches (e.g., when the classification head dimension changes between genus and species models). Phase 1 warmup is skipped when using transfer learning, as the backbone is already adapted.

3.5 Data Pipeline

3.5.1 Reference Database and Taxonomy

Reference genomes are drawn from the Human Gastrointestinal Reference genome database (HGR-UMGS) [39], comprising two sources merged at the genus level: the Human Gut Reference (HGR; `taxonomy_hgr.tab`) and the Unified Human Gastrointestinal Genome-derived Metagenome-Assembled Genomes Survey (UMGS; `taxonomy_umgs.tab`).

The database covers **1,535 genomes spanning 120 genera**; because the source taxonomy is provided only down to genus, each genome is treated as a distinct “species” and is identified by its genome ID (e.g., 11861_6_55). Read headers therefore encode the full label hierarchy as `>lbl|species_class|genome_id|genus_class|genus_name`, which is

parsed at load time to produce both 120-class genus and 1,535-class species labels. The same database underlies MetaTransformer [8], which simplifies cross-method comparison.



3.5.2 ART Read Simulation

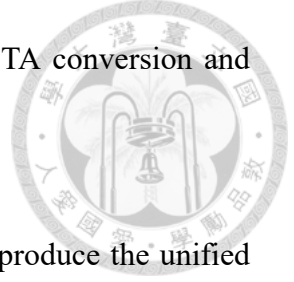
Training data consists of simulated metagenomic short reads generated from these reference genomes using the ART Illumina simulator [40], following the same simulation protocol as MetaTransformer [8]. ART is invoked in paired-end mode with the HiSeq 2500 (HS25) error profile, 150 bp read length, and fragment-length distribution $\mathcal{N}(\mu = 400, \sigma = 50)$ bp.

To prevent dataset composition from being driven by genome-length differences, reads are generated under a *coverage-normalised* schedule rather than a fixed-coverage schedule. The longest reference genome is assigned a coverage factor of $3\times$, and shorter genomes are assigned proportionally higher coverage so that, in expectation, every genome contributes the same number of reads to the merged FASTA. In practice, this yields per-genome read counts within a narrow band (~ 260 – 410 reads per genome) due to discretisation; the relative class composition is therefore determined by the number of genomes per genus rather than by total genus genome length.

Pipeline. The full pipeline executes in five stages:

1. ART generates paired-end reads for each genome FASTA under the coverage-normalised schedule above.
2. Per-genome FASTQ outputs are merged into a single FASTQ stream.

3. The stream is split into 64 chunks for parallel FASTQ→FASTA conversion and label injection.
4. The chunks are concatenated and shuffled with seq-shuf to produce the unified reads.fa (~47 GB, ~258M reads).
5. Balanced subsampling (Section 3.5.3) draws fixed-size subsamples (e.g., 50M, 5M, 500K) from reads.fa.



Reproducibility audit. We conducted an explicit audit of randomness sources across the full pipeline. The findings are summarised below.

Reproducible by construction. The balanced-subsampling stage (Section 3.5.3) seeds Python’s random module with seed=42, so all balanced subsamples (500K, 5M, 50M) used in this thesis are exactly reproducible from a given reads.fa. The train/validation split is computed by sklearn’s train_test_split with random_state=42 (Section 3.5), and is therefore also deterministic for a fixed input subsample.

Not bit-for-bit reproducible. ART is invoked without --rndSeed, so the underlying ~258M-read reads.fa would differ between fresh runs in individual read positions and per-base error realisations, although its aggregate statistical properties (per-genome coverage, label distribution, read-length distribution) are reproducible by construction. The training script likewise does not set torch.manual_seed, numpy.random.seed, or torch.cuda.manual_seed_all; torch.backends.cudnn.deterministic is left at its default (False); and the PyTorch DataLoader is constructed without worker_init_fn, so each worker’s numpy and random states are uncontrolled. Resume from a checkpoint restores model weights, optimiser state, and learning-rate scheduler state, but does not re-

store the PyTorch / CUDA / NumPy / Python RNG states, so a run resumed after a SLURM 48-hour timeout is not bit-for-bit identical to a hypothetical uninterrupted run.

Empirical impact. We did not perform repeated multi-seed runs of the full 50M-scale experiments due to compute constraints, but the residual non-determinism above is expected to produce run-to-run variation on the order of ± 0.1 – 0.3 pp in genus Top-1 accuracy, which is small relative to the effects discussed in this thesis (depth +14.02 pp from 1 to 16 layers; tokenization ~ 16 – 20 pp at matched or best-vs-best 13-mer vs. 6-mer). We acknowledge this as a methodological limitation and a candidate target for tightening in any follow-up study.

3.5.3 Balanced Subsampling

The full training dataset contains 258M reads with a highly skewed distribution across species. To mitigate class imbalance and enable controlled data scaling experiments, we implement a balanced subsampling strategy:

1. **Scan:** The full FASTA file is scanned to extract all read headers and their species labels.
2. **Allocate:** A target number of reads per species is computed as $n_{\text{target}} = N_{\text{total}} / K_{\text{species}}$, where N_{total} is the desired subsample size and K_{species} is the number of species.
3. **Cap and redistribute:** For species with fewer reads than n_{target} , all available reads are included. The deficit is redistributed proportionally among species with surplus reads.
4. **Sample:** Reads are randomly selected within each species to meet the per-species

allocation.



This procedure produces near-perfectly balanced subsamples. For a 50M subsample from 1,535 species, each species contributes $32,573 \pm 445$ reads.

3.5.4 Reverse-Complement Augmentation

During training, each read is replaced with its reverse complement with probability $p = 0.5$. This augmentation is applied on-the-fly during data loading, doubling the effective diversity of the training set without requiring additional storage.

3.5.5 Data Splitting

The balanced subsamples are split into training (90%) and validation (10%) sets at the *read* level—i.e., individual reads from the same genome may appear in both the training and validation splits. The split is stratified by species label so that every one of the 1,535 species is represented in both partitions and class proportions are preserved within each. For the hierarchical evaluation of Section 4.10.3, an additional, fully disjoint test set is drawn from the MetaTransformer validation reads (`val_reads.fa`); see Section 4.10.2 for the data-leakage audit motivating this choice and for empirical evidence that read-level (rather than genome-level) splitting does not lead to detectable memorisation in this study.



3.6 Evaluation Methodology

3.6.1 Metrics

We evaluate classification performance using the following metrics:

Micro accuracy (overall accuracy): The fraction of correctly classified reads. This metric is dominated by abundant classes and reflects real-world performance on typical samples.

Macro F1: The unweighted average of per-class F1 scores. This metric weights all classes equally, providing a measure of performance across the full taxonomic diversity including rare taxa.

Balanced accuracy: The average of per-class recall, equivalent to macro recall.

Top- k accuracy: The fraction of reads for which the correct class appears in the top k predictions. We report top-3, top-5, and top-10 accuracy.

F1=0 class count: The number of classes with zero F1 score (i.e., classes for which the model makes no correct predictions). This directly measures the “dead class” problem.

Train-validation gap: The difference between training accuracy and validation accuracy, serving as a proxy for overfitting.

3.6.2 Reverse-Complement Test-Time Augmentation

At inference, we employ RC TTA: each read is classified twice (forward and reverse complement), and the logits are averaged before applying softmax:

$$\hat{y} = \arg \max_k \frac{1}{2} [f_{\theta}(r) + f_{\theta}(\bar{r})] \quad (3.4)$$

where $f_{\theta}(r)$ and $f_{\theta}(\bar{r})$ are the logit vectors for the forward and reverse-complement reads, respectively. This ensembling exploits the biological strand symmetry of DNA and consistently improves accuracy.

3.6.3 Species-Level Evaluation Modes

For species-level classification (1,535 classes), we introduce two additional evaluation modes to decouple the contributions of genus-level and intra-genus discrimination:

Oracle-genus evaluation: Given the *true* genus label, the species logits are masked to only allow species belonging to that genus. This answers: “Given perfect genus identification, how well can the model discriminate species within each genus?”

Top- k genus routing: The genus classifier produces the top- k genus predictions. The candidate species set is the union of all species belonging to these k genera. Species logits are then masked to this candidate set. This simulates a realistic two-stage hierarchical classification pipeline.





3.7 Computational Considerations

3.7.1 Hardware

All experiments are conducted on the TWCC (Taiwan Computing Cloud) SLURM cluster equipped with NVIDIA H100 80 GB HBM3 GPUs. The SLURM partition enforces a 48-hour maximum wall time per job, necessitating checkpoint-resume support for long training runs.

3.7.2 Memory Optimization

Several techniques are employed to manage GPU memory:

- **Gradient checkpointing:** Recomputes intermediate activations during the backward pass rather than storing them, trading $\sim 30\%$ additional compute for $\sim 60\%$ memory reduction.
- **Mixed precision (bfloat16):** Reduces memory requirements for activations and gradients by approximately 50% compared to float32.
- **LoRA:** Keeps the majority of backbone parameters frozen, reducing the optimizer state memory from $\sim 4\text{GB}$ (full fine-tuning with AdamW) to $\sim 44\text{MB}$.

3.7.3 Resource Monitoring

To assess computational resource adequacy and identify bottlenecks, we implement a dedicated resource monitoring module that tracks:

- GPU VRAM usage (PyTorch allocated, nvidia-smi total, and peak values).

- GPU compute utilization (sampled every 30 seconds via `nvidia-smi`).
- System RAM usage (process RSS and peak).
- Per-epoch wall-clock time and throughput (reads per second).



These metrics are logged to `training_history.csv` and a comprehensive `resource_report.json` at the end of training.

Training and evaluation code, experiment configs, and the scripts that build the balanced read sets are at <https://github.com/m2lab-ntu/gfm-classifier>.

3.8 Summary

This chapter has described the complete methodology for token-level GFM classification of metagenomic reads. The framework combines a pre-trained Nucleotide Transformer v2 backbone with LoRA adaptation, an attention-pooling classification head, two-phase training with transfer learning support, balanced subsampling, and RC augmentation. From-scratch depth-sweep and same-architecture tokenizer variants reuse the same head and evaluation file so that Chapter 4 can separate depth, pre-training, and tokenization. The evaluation methodology encompasses multiple metrics at both genus and species levels, with specialized evaluation modes for dissecting hierarchical classification performance.



Chapter 4

Experiments and Results



This chapter presents the experimental setup and results. Frozen-embedding baselines and a 500K architecture search establish the token-level pipeline. Data scaling then shows that, *inside* the NT-v2 6-mer framework, volume dominates training tricks—and that the same volume does not raise a 13-mer ceiling. Depth-matched from-scratch controls and tokenization ablations separate pre-training from depth, and both from the tokenizer.

4.1 Experimental Setup

4.1.1 Dataset

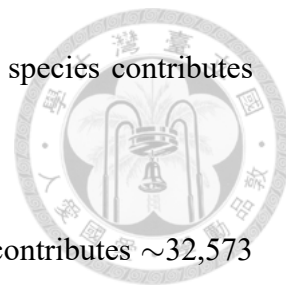
Training data is generated by simulating Illumina short reads (150 bp, HiSeq 2500 error profile) from the HGR-UMGS reference genome database [39] using the ART read simulator [40], following the same protocol as MetaTransformer [8]. The dataset covers:

- **120 genera** and **1,535 species** of human gut microorganisms.
- **258M total reads** across all species.
- Highly skewed distribution: the most abundant genus (*Collinsella*) accounts for 22.8% of reads, while the rarest genera contribute $<0.06\%$.

We conduct experiments at three data scales by subsampling from the full dataset:

- **500K** (imbalanced): The initial small-scale dataset preserving the natural class distribution. Train: 450K, validation: 50K.

- **5M** (balanced): Species-balanced subsample. Each of 1,535 species contributes $\sim 3,257$ reads. Train: 4.5M, validation: 500K.
- **50M** (balanced): Species-balanced subsample. Each species contributes $\sim 32,573$ reads. Train: 45M, validation: 5M.



Reads versus unique reads. The read counts above (and the data-scaling axis in Figure 4.3) refer to *training reads including duplicates*, which is the quantity the optimiser actually sees per epoch. The underlying source is heavily duplicated at the sequence level: the full read set contains 258.67M reads but only 42.64M *unique* sequences ($6.07\times$ redundancy), a property of the source simulation rather than of our balancing. Consequently the larger balanced pools contain fewer distinct reads than their nominal size: the 50M pool comprises 17.76M unique reads and the 250M pool 41.94M unique reads—the latter already approaching the 42.64M source ceiling. This duplication structure is one reason data scaling saturates: beyond ~ 50 M reads, additional “reads” are increasingly repeats the model has already seen.

4.1.2 Evaluation Protocol

All models are evaluated on a held-out validation set using the metrics described in Section 3.6. Unless otherwise noted, results are reported with RC TTA (Eq. 3.4). Depth-sweep and same-architecture tokenizer results in Sections 4.8 and 4.9 use a shared `clean_common` 100K file that is near-disjoint from the 50M training set (174 / 100,000 reads, 0.17%, share a `seq_id` with training). The original NT-Genus 67.07% figure is the 50M internal-validation RC-TTA number; re-scoring that checkpoint on `clean_common` gives 67.08%.



4.1.3 Hardware and Training Infrastructure

All experiments are run on NVIDIA H100 80 GB HBM3 GPUs on the TWCC SLURM cluster. The 48-hour SLURM time limit requires checkpoint-resume support for training runs exceeding this duration.

4.2 Phase 1: Frozen Embedding Baselines

Before developing the token-level approach, we establish baseline performance using precomputed, mean-pooled embeddings from frozen GFMs.

4.2.1 Setup

Embeddings are extracted once from each of several pre-trained GFMs. A two-layer MLP classifier is trained on the resulting fixed-dimensional vectors. We evaluate models from two families:

- **Evo-2 (7B params)** [23]: A large-scale model based on the StripedHyena architecture. We extract embeddings from layer 2.
- **Nucleotide Transformer v2 (500M params)**: ESM-2-based model. We extract embeddings from the last hidden layer.

4.2.2 Results

The NT-v2 backbone consistently outperforms Evo-2 despite having $14\times$ fewer parameters, suggesting that NT-v2’s multi-species pre-training provides more suitable representations for metagenomic classification. NT-v2 achieves 51.70% genus accuracy, which

Table 4.1: Frozen embedding baselines for genus classification (120 classes, 500K dataset).

Method	Genus Accuracy	Notes
Uniform random ($1/K$)	0.83%	Lower bound
Frequency baseline ($\sum p_k^2$)	8.11%	Predicting proportionally
Majority class (always <i>Collinsella</i>)	22.84%	Naïve baseline
Evo-2 L2 mean-pool + MLP	45.52%	
Evo-2 L2 + Focal Loss	46.95%	
NT-v2 mean-pool + MLP	51.70%	Best frozen baseline
NT-v2 mean-pool + MLP (RC-avg)	51.70%	With RC averaging

serves as our primary baseline.

4.3 Phase 2: Token-Level Classification with LoRA

We now present the token-level approach. Architecture and training-technique searches were run on 500K reads; they establish a plateau that motivates data scaling. The numbers that carry the argument of this chapter are at 5M and 50M.

4.3.1 Architecture and Training Techniques (v1–v7, 500K Dataset)

On 500K imbalanced reads, LoRA token-level training (v1) matches the frozen mean-pool baseline at 51.70% genus accuracy. Reverse-complement augmentation (v3) reaches 53.92% forward / 55.36% RC TTA. Class-weighted cross-entropy with label smoothing (v2) collapses to 27.87% (-23.8 pp), from the interaction of class weights and label smoothing on a 120-class imbalanced problem. Subsequent technique trials—shorter `max_length` (v4), logit adjustment (v5/v5b), RC consistency (v6/v7)—remain in a ± 0.5 pp band around the v3 RC-TTA plateau (Table 4.2). Logit adjustment with $\tau = 1.0$ is too aggressive (35.17%); $\tau = 0.3$ preserves micro accuracy while cutting F1=0 classes from 9 to 5. Combining RC consistency with logit adjustment (v6) is harmful; RC consistency

alone (v7) is comparable to v3.

Table 4.2: Architecture and training-technique experiments on the 500K dataset (genus classification). RC TTA results are shown where available. All technique variants after v3 stay within ± 0.5 pp of the v3 RC-TTA plateau except the failed logit-adjustment runs.

Ver.	Description	Fwd Acc	RC TTA	F1-macro	F1=0	Gap
v1	Token-level LoRA baseline	51.70%	—	—	—	—
v2	Class weights + LS (harmful)	27.87%	—	—	—	—
v3	RC aug, maxlen=128	53.92%	55.36%	25.82%	11	6.96%
v4	maxlen=32	53.75%	55.29%	25.70%	9	6.25%
v5	LA $\tau=1.0$	35.17%	—	—	—	—
v5b	LA $\tau=0.3$	52.29%	53.58%	26.53%	5	—
v6	RC consist.+LA	35.87%	38.19%	22.88%	—	—
v7	RC consist. only	54.10%	55.18%	25.10%	11	—

On v3, training accuracy continues to 62.7% while validation saturates near 54% (train-val gap 0% $\rightarrow$ 9% by epoch 35). Figure 4.1 shows the same gap shrinking as data scale increases: severe overfitting at 500K, small gap at 5M, and well-generalised training at 50M.

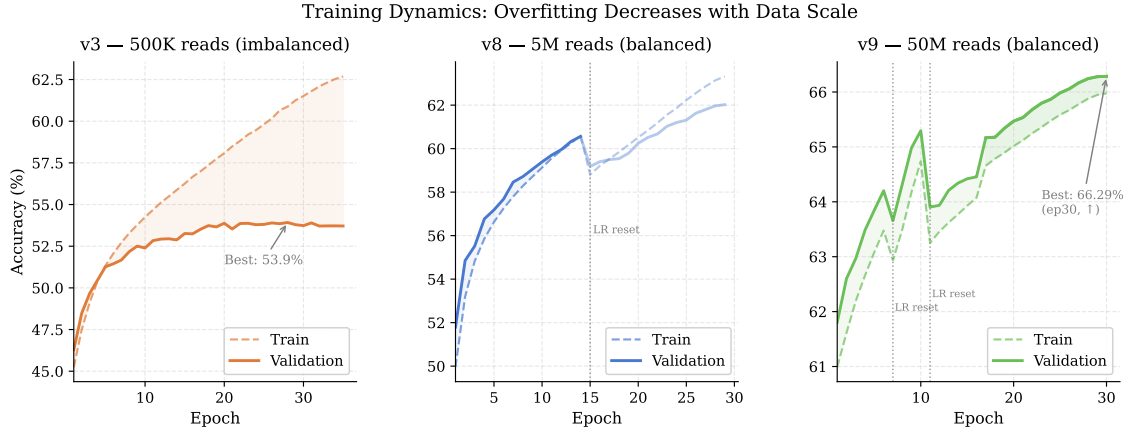


Figure 4.1: Training dynamics for v3 (500K, left), NT-Genus-5M (5M, centre), and NT-Genus (50M, right). Dashed lines show training accuracy; solid lines show validation accuracy. The shaded region highlights the train-validation gap. Dotted vertical lines mark learning-rate resets caused by SLURM timeout and job resumption; NT-Genus experienced two such resets (epochs 7 and 11) before the resume logic was corrected. Overfitting decreases systematically as data volume increases.

Conclusion: All training techniques on the 500K dataset yield at most ± 0.5 pp improvement in RC TTA accuracy among the non-failed runs (range: 53.58%–55.36%).

Inside this tokenizer and backbone, *data volume*, not architecture or training tricks, is the

bottleneck that the next section scales.



4.4 Data Scaling Experiments

Motivated by the accuracy plateau on 500K data and the comparison with MetaTransformer [8] (which trained on a large-scale balanced dataset trained on the full HGR-UMGS dataset of 2,505 species with coverage factor 4), we systematically scale training data.

4.4.1 NT-Genus-5M: 5M Balanced Reads

4.4.1.1 Setup

A 5M species-balanced subsample is drawn from the full 258M dataset. Each of 1,535 species contributes $\sim 3,257$ reads, yielding near-perfect class balance. The v3 architecture and training configuration are retained (batch size 32, gradient accumulation 2, effective batch size 64), and training proceeds for 30 epochs.

Due to the 48-hour SLURM time limit, training required two jobs: the initial run completed 14 of 20 epochs before timeout, and a resumed run completed 16 additional epochs (epochs 15–30) using the `--resume` mechanism. A third job performed final evaluation.

4.4.1.2 Results

The results reveal clear log-linear scaling behaviour:

1. **Accuracy:** Each $10\times$ data increase yields a substantial but diminishing gain in RC TTA accuracy: $+7.76$ pp (500K $\rightarrow$ 5M) and $+4.02$ pp (5M $\rightarrow$ 50M). The sub-linear

Table 4.3: Genus classification results across data scales: v4 (500K), NT-Genus-5M (5M balanced), and NT-Genus (50M balanced).

Metric	v4 (500K)	NT-Genus-5M	$\Delta_{v4 \rightarrow 5M}$	NT-Genus (50M) [†]	$\Delta_{5M \rightarrow 50M}$
Micro Accuracy (fwd)	53.75%	62.02%	+8.27 pp	66.29%	+4.27 pp
Micro Accuracy (RC TTA)	55.29%	63.05%	+7.76 pp	67.07%	+4.02 pp
Balanced Accuracy	25.23%	33.35%	+8.12 pp	39.91%	+6.56 pp
F1 (weighted)	52.51%	61.06%	+8.55 pp	65.61%	+4.55 pp
F1 (macro)	25.70%	37.97%	+12.27 pp	44.82%	+6.85 pp
Top-3 Accuracy	—	81.70%	—	84.40%	+2.70 pp
Top-5 Accuracy	82.91%	88.08%	+5.17 pp	90.03%	+1.95 pp
Top-10 Accuracy	91.29%	94.23%	+2.94 pp	95.25%	+1.02 pp
F1=0 classes	9	2	-7	0	-2
Train-Val Gap	6.25%	1.30%	-4.95 pp	<1%	—

[†] NT-Genus final metrics (epoch 30, converged).

trend is consistent with logarithmic scaling laws.

2. **Macro F1:** Data scaling disproportionately benefits rare classes: +12.27 pp at the first step and +6.85 pp at the second. At 50M reads, all 120 genera have non-zero F1 for the first time.
3. **F1=0 classes:** The number of unclassifiable genera drops from 9 to 2 (NT-Genus-5M) and finally to 0 (NT-Genus), meaning every genus in the dataset is correctly predicted for at least some reads.
4. **Overfitting:** The train-validation gap shrinks dramatically from 6.25% (v4) to 1.30% (NT-Genus-5M) to <1% (NT-Genus), confirming that the model transitions from data starvation to well-generalised training as data volume increases.

4.4.1.3 Training Dynamics

The temporary accuracy drop at epoch 15 is caused by the learning rate scheduler resetting to its warmup phase after resumption. The model recovers within 3–4 epochs and eventually surpasses the pre-interruption performance.

Table 4.4: Training dynamics for NT-Genus-5M (5M balanced, 30 epochs). The gap between training and validation accuracy remains small throughout, in sharp contrast to the 500K experiments.

Epoch	Train Acc	Val Acc	Train-Val Gap	Notes
1	49.96%	51.80%	-1.8%	
5	55.97%	57.04%	-1.1%	
10	58.93%	59.42%	-0.5%	
14	60.60%	60.56%	+0.04%	48h limit
15	58.80%	59.18%	-0.4%	Resumed (LR reset)
20	61.82%	60.59%	+1.2%	
25	62.89%	61.77%	+1.1%	
29*	63.32%	62.02%	+1.3%	Best epoch

4.4.2 NT-Genus Model: 50M Balanced Reads

Building on NT-Genus-5M, we scale training data by another $10\times$ to 50M balanced reads. The batch size is increased from 32 to 256 (with gradient accumulation reduced from 2 to 1, yielding an effective batch size of 256) to improve GPU utilization. The model is initialized from NT-Genus-5M’s best checkpoint using `--init_from` (transfer learning mode).

Table 4.5: Configuration comparison between NT-Genus-5M and NT-Genus.

Parameter	NT-Genus-5M	NT-Genus
Training reads	4.5M	45M
Reads per species	$\sim 3,257$	$\sim 32,573$
Batch size	32	256
Gradient accumulation	2	1
Effective batch size	64	256
Epochs	30	30 (converged)
Weight initialization	Random	v8 best.pt
Est. time per epoch	$\sim 3h$	$\sim 5-6h$

4.4.2.1 Results

Scaling from 5M to 50M balanced reads yields a further +4.02 pp improvement in RC TTA accuracy ($63.05\% \rightarrow 67.07\%$). Critically, the number of F1=0 classes drops to

Table 4.6: Comparison between NT-Genus-5M and NT-Genus (50M) for genus classification (120 classes).

Metric	NT-Genus-5M	NT-Genus	Δ
Micro Accuracy (fwd)	62.02%	66.29%	+4.27 pp
Micro Accuracy (RC TTA)	63.05%	67.07%	+4.02 pp
Balanced Accuracy	33.35%	39.91%	+6.56 pp
F1 (weighted)	61.06%	65.61%	+4.55 pp
F1 (macro)	37.97%	44.82%	+6.85 pp
Top-3 Accuracy	81.70%	84.40%	+2.70 pp
Top-5 Accuracy	88.08%	90.03%	+1.95 pp
Top-10 Accuracy	94.23%	95.25%	+1.02 pp
F1=0 classes	2	0	−2
Best epoch	29	30 (converged)	—

zero, indicating that all 120 genera are now correctly classified in at least some reads.

The marginal return diminishes relative to the previous $10\times$ scaling step: 500K $\rightarrow$ 5M yielded +7.76 pp while 5M $\rightarrow$ 50M yields +4.02 pp for the same multiplicative data increase, consistent with a logarithmic scaling relationship. This sub-linear trend is expected under log-linear scaling laws and suggests that continued data scaling alone will not close the remaining gap with MetaTransformer.

Training progress and LR restart incidents. NT-Genus training required multiple SLURM jobs due to the 48-hour time limit. Before a bug fix was applied to the checkpoint-resume logic, two job restarts inadvertently reset the cosine learning-rate scheduler to its initial warmup phase, causing temporary accuracy dips: −0.54 pp at epoch 7 and −1.39 pp at epoch 11. In both cases the model recovered within 3–5 epochs and surpassed the pre-dip performance level. The underlying issue—`EarlyStopping.best` being silently set to `None` from an incomplete checkpoint key—was corrected in the resume logic; from epoch 17 onward the scheduler state is correctly restored and the learning rate continues its cosine decay without interruption.

NT-Genus training completed at epoch 30 (April 2026), reaching a forward validation accuracy of **66.29%** and RC TTA accuracy of **67.07%** (Figure 4.2). The learning curve plateaued between epochs 26 and 30 as the cosine schedule reached its minimum, indicating convergence.

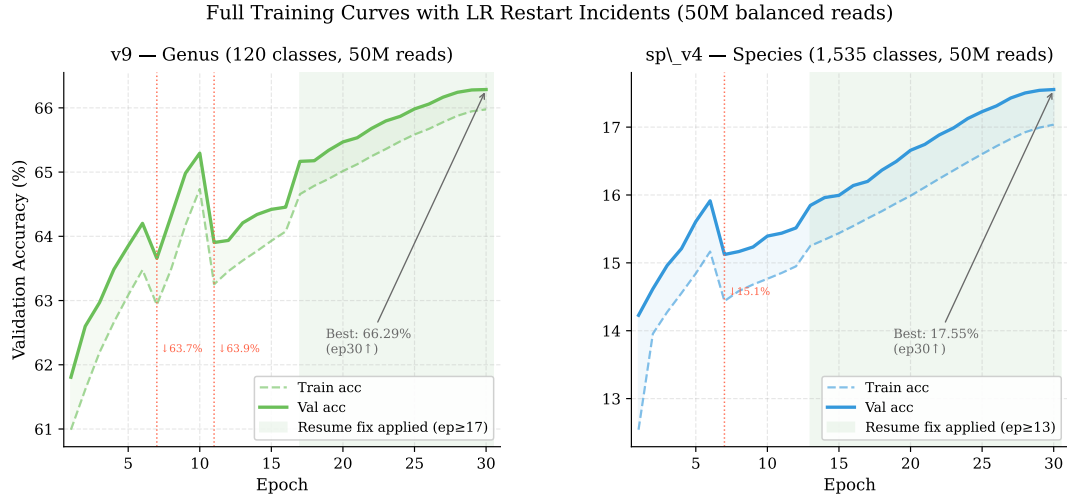


Figure 4.2: Full validation accuracy curves for NT-Genus (genus, left) and NT-Species (species, right) over all completed epochs. Red dotted vertical lines mark epochs where a SLURM job restart incorrectly reset the cosine LR scheduler, causing a temporary accuracy dip. The green shaded region indicates epochs trained after the resume bug fix was applied, during which the LR schedule correctly continues its cosine decay.

4.4.3 Data Scaling Analysis

Table 4.7: Data scaling analysis: genus classification performance across data volumes. The 250M entry is the warm-started model (initialised from the 50M checkpoint and held out of the log-linear fit); the from-scratch 250M variant instead reaches 64.84%. Only accuracy is reported for the 250M saturation experiment.

Data Volume	RC TTA Acc	F1-macro	F1=0	Gap
500K (imbalanced)	55.29%	25.70%	9	6.25%
5M (balanced)	63.05%	37.97%	2	1.30%
50M (balanced)	67.07%	44.82%	0	~0.7%
250M (balanced, warm-start)	67.29%	—	—	—
MetaTransformer (HGR-UMGS full)	~98%	—	—	—

The data scaling analysis reveals a clear relationship between training data volume and classification performance *inside the NT-v2 6-mer framework*. The transition from

500K to 5M reads ($10\times$ increase) yields +7.76 pp in accuracy and +12.27 pp in macro F1. Meanwhile, all training technique ablations on the 500K dataset (v3–v7) produced at most ± 0.5 pp variation. Data volume, not architecture or training tricks, is therefore the primary determinant of 6-mer accuracy up to 50M. It is not the determinant of the residual gap to MetaTransformer: the 250M run below and Section 4.9 show that gap is tokenizer-bound.

The published MetaTransformer genus recall of 98.3% used a different data budget and tokenizer; it is not evidence that we simply need more 6-mer reads.

Figure 4.3 plots genus RC TTA accuracy against training data volume on a logarithmic scale, with a log-linear fit to our three data points. The fitted curve projects that closing the gap with MetaTransformer would require training data well beyond our 50M-read scale, consistent with diminishing returns under logarithmic scaling.

Beyond 50M: the 6-mer model saturates, but the 13-mer model does not. The projection above tentatively attributes the residual gap to insufficient data. To test this directly, we scaled the balanced subsample by a further $5\times$ to 250M reads and trained two variants on a $64\times$ H200 DDP cluster: a warm-started model initialised from the 50M NT-Genus checkpoint (mirroring the $5\text{M} \rightarrow 50\text{M}$ transfer) and a from-scratch model. Evaluated on an independent, read-disjoint test set under the identical RC-TTA protocol—on which the 50M NT-Genus model reproduces its 67.07% to within 0.05 pp—the 250M warm-started model attains only **67.29%** genus Top-1 (RC TTA), a +0.22 pp change over 50M despite the $5\times$ data increase, while the from-scratch 250M model is in fact lower at 64.84%. For the NT-v2 6-mer model, data scaling therefore *saturates* beyond 50M rather than continuing along the log-linear trend.

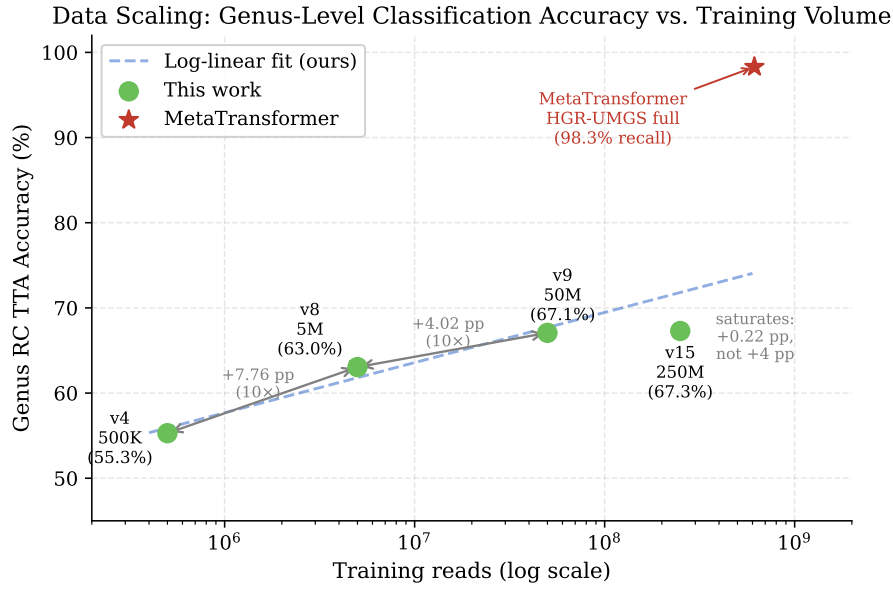
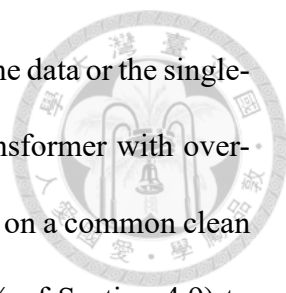


Figure 4.3: Genus RC TTA accuracy versus training data volume (log scale). The horizontal axis counts *training reads including duplicates* (the per-epoch sample count); the corresponding *unique*-read counts are smaller (50M→17.8M, 250M→41.9M unique, against a 42.6M source ceiling). Green circles mark our results at three data scales; the red star marks MetaTransformer’s reported genus recall. The blue dashed line is a log-linear fit to our three data points. Per-10× accuracy gains diminish from +7.76 pp (500K→5M) to +4.02 pp (5M→50M). The fourth green circle (v15, 250M) continues the same warm-started series but is held out of the fit; at 67.29% it falls well below the ~71% the log-linear fit would project at that scale—the trajectory flattens off its own projection, directly demonstrating that accuracy saturates beyond 50M rather than continuing along the trend. The MetaTransformer star marks reported genus recall (98.3%); its x-position is based on training throughput (300K steps × batch 2,048) and is not directly comparable to our dataset-size axis.



Crucially, this saturation is a property of the *tokenization*, not of the data or the single-genome-per-species reference. Trained on the *same* reads, MetaTransformer with overlapping 13-mer tokenization continues to improve sharply with scale: on a common clean held-out pool its genus Top-1 rises from 87.5% at 50M (the 87.42% of Section 4.9) to **98.7%** at 250M—and from 83.0% to 98.5% on a stricter “leftover” pool constructed to be sequence-ID-disjoint from both training sets, which rules out leakage. The identical $5\times$ data increase that moves the 6-mer model by +0.22 pp moves the 13-mer model by $\sim +11$ pp. The additional reads are therefore not redundant in any absolute sense: they carry discriminative signal that a non-overlapping 6-mer representation cannot resolve but an overlapping 13-mer one can. What saturates at 50M is the *information ceiling of the 6-mer tokenizer*, not the data.

We therefore conclude that whether additional training data helps is governed by tokenization: the remaining gap to MetaTransformer is *not* closable by additional data within the NT-v2 6-mer framework, whereas an overlapping long- k tokenizer converts the same reads into near-perfect genus assignment. We caution, however, that the 98.7% figure reflects *near-complete closed-set coverage of the 1,535 reference genomes’ 13-mer content*—at 250M reads the overlapping 13-mer model approaches a lookup over the known genomes—and does *not* measure generalisation to novel, unseen genomes. The operative ceiling for our setting is thus tokenization (Section 4.9) rather than data volume.

4.4.4 Balancing Axis: Species-Balanced vs. Genus-Balanced Training

The scaling experiments above are species-balanced (each of the 1,535 species contributes equally). A natural alternative is to balance at the *genus* level (each of the 120

genera contributes equally), which up-weights species-poor genera. To isolate the effect of the balancing axis alone, we trained two NT-v2 6-mer models on the *same* 17.6M-read budget, differing only in this choice, and evaluated both on a common natural-abundance read pool. Table 4.8 reports the result across three complementary metrics: per-read micro accuracy (the rate a randomly drawn read is classified correctly, the operationally relevant quantity for real samples), per-genus flat accuracy (macro-averaged over the 120 genera, treating each genus equally), and the Pearson correlation of estimated versus true relative abundance on real samples.

Table 4.8: Effect of the training balancing axis at a fixed 17.6M-read budget (same reads, balancing differs only in whether species or genera are equalised). Genus-balancing trades a +10 pp gain in macro/flat accuracy for a −23 pp loss in per-read accuracy and a sharp drop in real-sample abundance fidelity.

Balancing	Per-read acc.	Per-genus flat acc.	Sample-level r
Species-balanced (sbal)	60.4%	33.1%	0.993
Genus-balanced (gbal)	37.2%	43.5%	0.862

Two findings follow. First, genus-balancing is *counterproductive for real metagenomics*: although it lifts the macro/flat accuracy that weights rare genera equally (33.1% → 43.5%, +10.4 pp), it does so by sacrificing exactly the reads that dominate real samples—per-read accuracy collapses from 60.4% to 37.2% (−23.2 pp) and the relative-abundance correlation degrades from $r = 0.993$ to $r = 0.862$. Because real samples are dominated by abundant genera, the per-read and abundance metrics are the ones that matter operationally, and on both the species-balanced model is decisively better. Second, the per-genus flat accuracy plateaus at ~42–43% *even when the training distribution is explicitly genus-balanced*. This ceiling is therefore not a fixable artefact of class imbalance; it reflects the intrinsic difficulty of resolving 120 genera from 150 bp reads under 6-mer tokenization—consistent with the train-fit ceiling of Section 4.9.

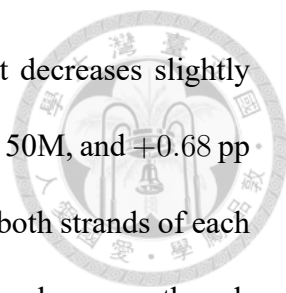
4.5 RC Test-Time Augmentation Analysis

Table 4.9: Effect of RC TTA across experiments. RC TTA consistently improves accuracy at the cost of $2\times$ inference time.

Experiment	Fwd Only	RC TTA	Δ
v3 (500K genus)	53.92%	55.36%	+1.44 pp
v4 (500K genus)	53.75%	55.29%	+1.54 pp
v5b (500K, LA $\tau=0.3$)	52.29%	53.58%	+1.29 pp
v7 (500K, RC consist.)	54.10%	55.18%	+1.08 pp
NT-Genus-5M (5M genus)	62.02%	63.05%	+1.03 pp
NT-Genus (50M genus)	66.29%	67.07%	+0.78 pp
NT-Genus (250M genus, warm-start)	66.61%	67.29%	+0.68 pp
Shallow-Genus (50M genus, shallow)	53.80%	53.88%	+0.08 pp

RC TTA provides a consistent +1.0 to +1.5 pp accuracy improvement across all experiments, regardless of the training configuration or data scale. Several observations merit discussion:

1. **Biological basis:** As discussed in Section 2.5.1, a metagenomic read and its reverse complement encode identical biological information. RC TTA exploits this strand symmetry by treating the two orientations as a two-view ensemble of the same underlying genomic entity [31].
2. **Variance reduction:** The consistent improvement across all settings is explained by standard ensemble theory: averaging two correlated but independently noisy estimates of the same ground-truth label reduces prediction variance, even when both estimates come from the same model.
3. **Diminishing returns with RC consistency training:** v7 (RC consistency, +1.08 pp) shows less benefit than vanilla models, because RC consistency training already encourages $f(s) \approx f(\bar{s})$ during training, leaving less residual strand asymmetry for RC TTA to correct.

- 
4. **Scale-dependent diminishing returns:** The RC TTA benefit decreases slightly with data scale: +1.44 pp at 500K, +1.03 pp at 5M, +0.78 pp at 50M, and +0.68 pp at 250M (NT-Genus). Larger training sets expose the model to both strands of each locus more often, gradually reducing systematic strand-directional errors—though a non-trivial asymmetry persists even at 50M reads.
 5. **Near-zero benefit for randomly initialized models:** Shallow-Genus (shallow Transformer, random initialization) shows only +0.08 pp from RC TTA. Without pre-training on a large genomic corpus, the model does not develop systematic strand-direction preferences, leaving little strand asymmetry for RC TTA to correct. This further supports the view that RC TTA primarily corrects pre-training-induced strand biases rather than stochastic inference noise.

The $2\times$ inference cost is the primary trade-off. For offline evaluation this is acceptable; for latency-sensitive deployment, RC-equivariant architectures [33] or RC consistency regularization [32] may be preferred alternatives.

Figure 4.4 summarises RC TTA gains across all experiments, highlighting the near-zero benefit for the randomly initialized Shallow-Genus model.

4.6 Computational Resource Analysis

4.6.1 Observed Resource Usage

4.6.2 Bottleneck Analysis

The hardware is substantially over-provisioned for this workload: the H100 80 GB GPU runs at $<50\%$ VRAM utilization even with batch size 256, and system RAM usage is well within limits. The primary operational bottleneck is the 48-hour SLURM wall

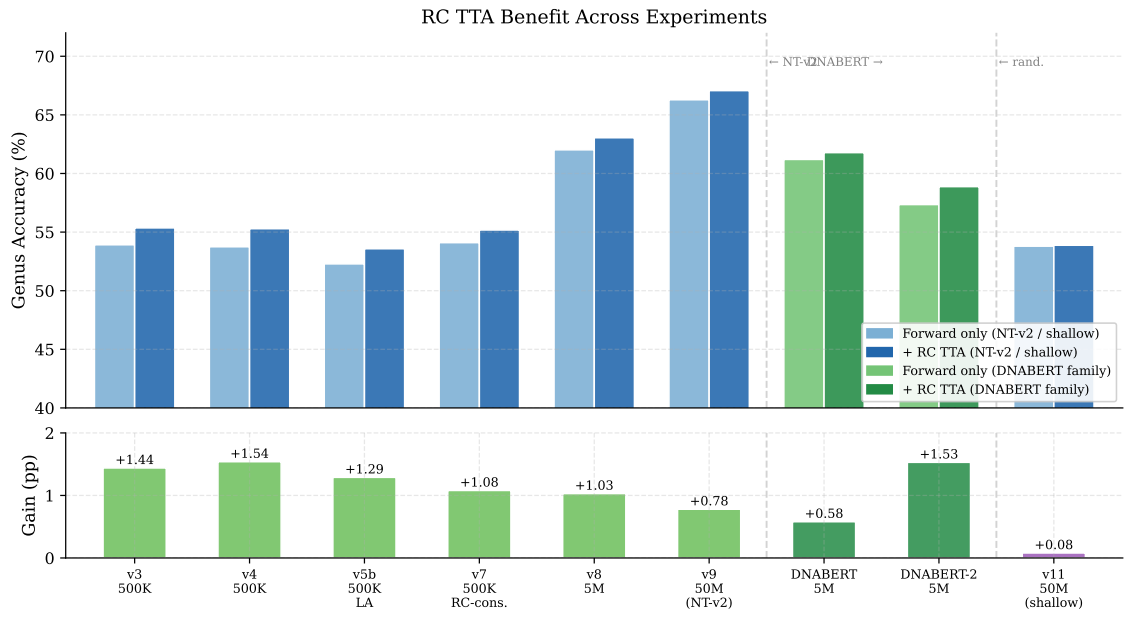


Figure 4.4: RC TTA benefit across experiments. Top: forward-only vs. RC TTA accuracy (blue: NT-v2 backbone; green: DNABERT family; right of second dashed line: random init). Bottom: absolute gain in percentage points. NT-v2-based models gain +0.77–+1.54 pp consistently. DNABERT gains only +0.58 pp, likely because its overlapping 6-mer tokenization is inherently more RC-symmetric than NT-v2’s non-overlapping 6-mer. DNABERT-2 (BPE tokenization) gains +1.53 pp, similar to early NT-v2 experiments. The randomly initialized Shallow-Genus gains only +0.08 pp, confirming that the benefit primarily corrects pre-training-induced strand biases.

Table 4.10: Computational resource usage across experiments.

Metric	v3 (500K)	NT-Genus-5M	NT-Genus
GPU	V100 32GB	H100 80GB	H100 80GB
Batch size	32	32	256
VRAM peak (training)	10.4 GB	~4.7 GB	~5 GB
VRAM utilization	32.5%	5.5%	<7%
Train throughput	1,128 r/s	~1,800 r/s	~1,825 r/s
Inference throughput	—	~5,100 r/s	~5,170 r/s
Epoch time	~6.5 min	~6.8h	~6.9h
Total training	11.5h	~90h	~207h (30 ep.)
SLURM jobs needed	1	3	4+

Table 4.11: Resource bottleneck assessment on TWCC H100 nodes.

Resource	Status	Assessment
GPU VRAM (80 GB)	<50% utilized	No upgrade needed
GPU compute (H100)	Adequate	No upgrade needed
System RAM (128 GB)	<50% utilized	No upgrade needed
Storage	<10 GB per experiment	No upgrade needed
SLURM time (48h)	Primary bottleneck	Requires resume or longer limits

time limit, which necessitates multiple job submissions with checkpoint resumption for large-scale experiments.



4.7 Comparison with MetaTransformer

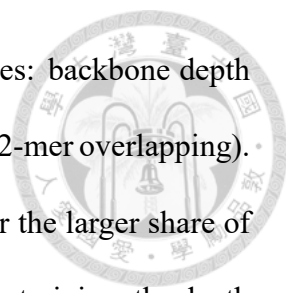
Table 4.12: Comparison between our approach and MetaTransformer [8].

Aspect	NT-Genus	MetaTransformer
Backbone	NT-v2 (498M)	Custom Transformer ($\sim 5\text{M}$ encoder [†])
Tokenization	6-mer (pre-trained)	12-mer (learned)
Pre-training	Yes (850 genomes, MLM)	No (trained from scratch)
Trainable params	5.54M (LoRA)	$\sim 5\text{M}$ encoder [†] (all)
Training data	50M balanced	HGR-UMGS full (2,505 sp., coverage 4)
Genus accuracy	67.07% (RC TTA)	98.3% recall (val)
Data source	HGR-UMGS + ART	HGR-UMGS + ART (same)

[†]Encoder parameters only; the embedding table is not counted here. For the overlapping 13-mer (stride 1) configuration, the embedding table alone is $\sim 2.1\text{B}$ parameters (see the parameter-accounting caveat below).

Despite using a foundation model $100\times$ larger than MetaTransformer, our model (at 50M training reads) substantially underperforms MetaTransformer (trained on the full HGR-UMGS dataset). The key differences are:

1. **Training data volume:** MetaTransformer uses the full HGR-UMGS dataset with coverage 4 vs. our 50M balanced reads. Inside the 6-mer framework each $10\times$ data increase yields +3–8 pp with diminishing returns, but the 250M run (Section 4.4) shows that extra 6-mer data does not close this gap.
2. **Training paradigm:** MetaTransformer trains all encoder parameters from scratch ($\sim 5\text{M}$; the embedding table is additional—see parameter-accounting caveat below), while we fine-tune only 5.54M LoRA parameters of a 498M-parameter model.
3. **Tokenization:** MetaTransformer uses 12-mer tokenization (capturing longer sequence patterns per token), while NT-v2 uses 6-mers.



This comparison is confounded by three simultaneous differences: backbone depth (29 vs. 1 layer), pre-training (yes vs. no), and tokenization (6-mer vs. 12-mer overlapping). Sections 4.8 and 4.9 separate these factors. Tokenization accounts for the larger share of the gap. The one-layer versus 29-layer comparison does *not* isolate pre-training; the depth sweep in Section 4.8 does.

The “~5M parameters” figure counts only the encoder. The ~5M parameter count attributed to MetaTransformer throughout this comparison refers to its *transformer encoder* alone; it omits the token-embedding table, which for large- k tokenization dominates the parameter budget. A learned k -mer embedding assigns a distinct vector to every k -mer observed in training, so the embedding matrix grows with the realized vocabulary. For the overlapping 13-mer (stride 1) configuration that attains 87.42% at 50M and 98.7% at 250M, the realized vocabulary reaches ~33.5M distinct k -mers; at the embedding width $d_{\text{model}}=64$, the embedding matrix alone is $\sim 33.5\text{M} \times 64 \approx 2.1$ billion parameters—roughly $4\times$ the size of the entire 498M-parameter NT-v2 backbone. The 13-mer model is therefore parameter-light only in *compute* (a single shallow encoder block over short token sequences), while being parameter-heavy in *storage* (a multi-billion-entry lookup table). This is intrinsic to the approach: its discriminative power comes precisely from giving each long k -mer its own vector, which is also what inflates the table. A fair efficiency comparison must therefore weigh MetaTransformer’s small compute footprint against this large embedding-memory cost, rather than citing the ~5M encoder figure as the model’s total size.

4.8 Backbone Ablation: Depth, Then Pre-training

A one-layer randomly initialized Transformer with the same 6-mer tokenizer and attention-pooling head as NT-v2 (Section 3.2.4) is the comparison originally used to “isolate pre-training.” It does not: encoder depth (29 vs. 1) and width ($d=1024$ vs. $d=128$) change at the same time as pre-training. We keep that comparison as a confounded baseline, then vary only depth.

4.8.1 Confounded baseline: 29-layer NT-v2 vs. one-layer from scratch

Table 4.13: Confounded backbone comparison: NT-v2 (NT-Genus) vs. one-layer Transformer (Shallow-Genus). Tokenizer, head, and 50M reads are shared; depth, width, and pre-training are not.

Component	NT-Genus	Shallow-Genus
Tokenizer	6-mer (NT-v2)	6-mer (NT-v2)
Encoder	29-layer, $d=1024$	1-layer, $d=128$
Pre-training	MLM on 850 genomes	None
Trainable params	5.54M (LoRA)	~200K (all)
Classification head	Attention pool (4 heads)	Attention pool (4 heads)
Training data	50M balanced	50M balanced
Batch size	256	512

Table 4.14: Confounded backbone results on the original 50M splits: pre-trained NT-v2 (NT-Genus) vs. one-layer Transformer (Shallow-Genus). The -13.19 pp RC-TTA gap is real; it is not an isolated pre-training effect.

Metric	NT-Genus	Shallow-Genus	Δ
Micro Accuracy (fwd)	66.29%	53.80%	-12.49 pp
Micro Accuracy (RC TTA)	67.07%	53.88%	-13.19 pp
Balanced Accuracy	39.91%	22.28%	-17.63 pp
F1 (macro)	44.82%	25.78%	-19.04 pp
F1=0 classes	0	3	+3
RC TTA gain	+0.78 pp	+0.08 pp	—
Best epoch	30 / 30	15 / 15	—
Inference throughput	~5,170 r/s	~750,000 r/s	$\approx 145\times$
VRAM (training)	~5 GB	~0.07 GB	—

NT-Genus outperforms Shallow-Genus by **13.19 pp** RC TTA (67.07% vs. 53.88%) on

the original splits. Macro F1 drops more than micro accuracy (-19.04 pp vs. -13.19 pp). RC TTA is nearly ineffective for the randomly initialized model ($+0.08$ pp vs. $+0.78$ pp). Shallow-Genus is $\sim 145\times$ faster at inference. None of these observations isolates pre-training from depth. Figure 4.5 shows the same confounded pair.

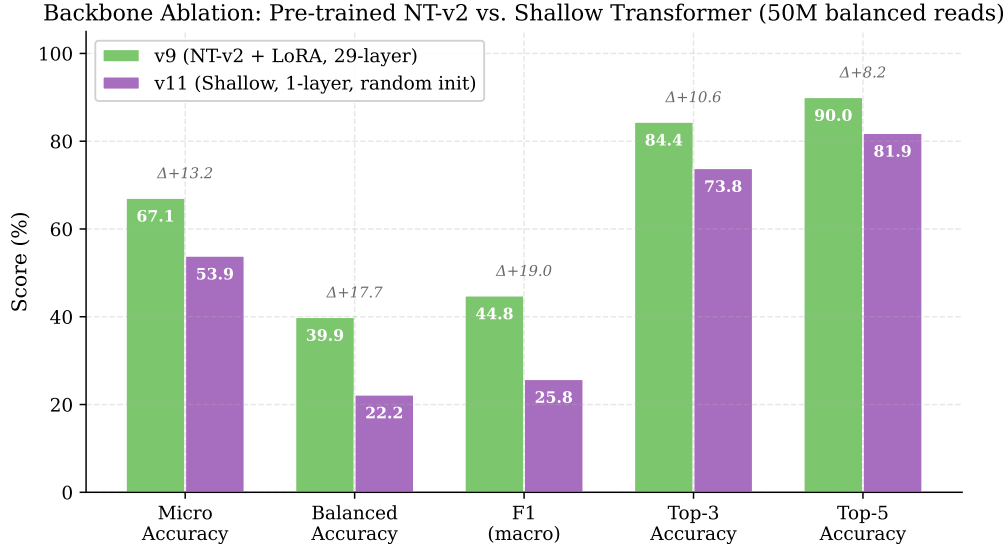


Figure 4.5: Confounded backbone comparison: pre-trained NT-v2 + LoRA (NT-Genus, green) vs. one-layer randomly initialized Transformer (Shallow-Genus, purple), both trained on 50M balanced reads. The gaps are real and include depth and width, not pre-training alone.

4.8.2 Depth sweep at matched 6-mer tokenization

Table 4.15 holds the tokenizer (NT-v2 non-overlapping 6-mer), the 50M gut-read pool, the attention-pooling head, 15 training epochs, RC TTA, and the `clean_common` 100K file fixed, and varies only encoder depth for the from-scratch models (Section 3.2.4). NT-v2 LoRA is the pre-trained reference on the same file.

Depth alone accounts for **+14.02 pp** ($53.92\% \rightarrow 67.94\%$). That is larger than the $+13.19$ pp previously attributed to pre-training. The 16-layer from-scratch model *exceeds* NT-v2 by $+0.86$ pp micro and $+3.7$ pp balanced accuracy with $129\times$ fewer parameters. A 3.88M-parameter model trained 100% from scratch therefore matches the 6-mer ceiling of

Table 4.15: Depth sweep, non-overlapping 6-mer, 50M gut reads, RC TTA on the same `clean_common` 100K file. From-scratch models train 100% of parameters. NT-v2 is LoRA-tuned ($\sim 1\%$ of 500M).

Model	Params	Top-1	Balanced acc
1 layer, from scratch	0.90M	53.92%	0.220
8 layers, from scratch	2.29M	65.44%	0.388
16 layers, from scratch	3.88M	67.94%	0.435
NT-v2, 29 layers, pre-trained + LoRA	500M	67.08%	0.398

a 500M LoRA-tuned GFM; the ceiling is not an artefact of updating only $\sim 1\%$ of NT-v2. Neither from-scratch run had early-stopped at epoch 15, so the from-scratch numbers are lower bounds.

The one-layer re-run on `clean_common` (53.92%) agrees with the original Shallow-Genus 53.88% on the internal split. NT-v2 on `clean_common` (67.08%) agrees with NT-Genus 67.07%. The two pools match; the attribution does not.

For the 8-layer model, internal validation rose $0.634 \rightarrow 0.654$ from epoch 9 to 15 while `clean_common` stayed at 65.44%: the last six epochs memorised duplicated reads with no generalisation gain, consistent with the $6.07\times$ sequence-level redundancy in Section 4.1. Excluding the 0.17% of `clean_common` reads that share a `seq_id` with training changes 65.44% to 65.43%.

4.8.3 Data-efficiency crossover at matched depth

The 50M result says pre-training does not raise the 6-mer ceiling. Table 4.16 asks whether it still helps when labels are scarce. A 16-layer from-scratch 6-mer model is trained on the same 5M gut reads used for NT-v2 v8 and scored on `clean_common` with RC TTA.

At 5M, going from 8 to 16 layers buys +0.49 pp; at 50M the same step is +2.50 pp.

Table 4.16: Depth-matched 6-mer comparison at 5M gut reads (RC TTA). The 16-layer from-scratch model is scored on `clean_common`. NT-v2 v8’s 63.05% is its recorded internal-validation figure (that checkpoint was not re-scored on `clean_common`); models we could check agree across the two pools (8-layer 0.6539 / 65.44%, 16-layer 0.6772 / 67.94%, NT-v2 v9 0.6707 / 67.08%), and the 16-layer 5M run itself scored 0.5411 on both.

Model (5M, 6-mer)	Params	Top-1
8-layer from scratch	2.29M	53.62%
16-layer from scratch	3.88M	54.11%
NT-v2 v8, pre-trained + LoRA	500M	63.05%

Depth needs data to pay off, so extra from-scratch depth would not close the low-data gap.

Combined with Table 4.15:

Table 4.17: Depth-matched pre-training contribution (16-layer from-scratch vs. NT-v2, non-overlapping 6-mer). Pre-training buys data efficiency, not a higher 50M ceiling.

Training reads	16-layer scratch	NT-v2	Net pre-training
5M	54.11%	63.05%	+8.94 pp
50M	67.94%	67.08%	−0.86 pp

Soil 50M comparisons of NT-v2 against a one-encoder-block MetaTransformer remain depth-confounded in the same way as Table 4.14; they are not reported here as evidence for pre-training. Depth-matched soil controls were not completed.

4.9 Tokenization Ablation: MetaTransformer at 50M Scale

The depth sweep (Section 4.8) shows that, at matched 6-mer tokenization, pre-training does not raise the 50M ceiling. Tokenization is the remaining design difference relative to MetaTransformer. We first train the MetaTransformer architecture [8] on the same 50M balanced dataset under six tokenization configurations ($k \in \{6, 12, 13\}$, $\text{stride} \in \{1, k\}$), using the original MetaTransformer codebase on the Taiwania-2 cluster (V100 32 GB GPU). All configurations are evaluated on the same stratified 90/10 val split (5M reads). The 13-mer model uses `embed_dim` = 64 to fit within V100 memory, consistent with the

original MetaTransformer paper’s 13-mer configuration [8]. A same-architecture 16-layer comparison in our codebase follows (Section 4.9).

Table 4.18: Tokenization ablation (genus-level, 120 classes): NT-v2 + LoRA vs. MetaTransformer under six tokenization configurations. All MetaTransformer models are trained from scratch on 50M balanced reads. RC TTA is unavailable for MetaTransformer.

Model	Tokenization	Tokens/read	Top-1 Acc	Macro F1
MetaTransformer	13-mer, stride=1	138	87.42%	0.7936
MetaTransformer	12-mer, stride=1	139	78.83%	0.6531
NT-Genus	6-mer, stride=6	25	67.07% [†]	—
NT-Genus	6-mer, stride=6	25	66.29%	0.4384
MetaTransformer	6-mer, stride=1	145	48.87%	0.1981
MetaTransformer	6-mer, stride=6	25	47.00%	0.1682
MetaTransformer	12-mer, stride=12	12	40.66%	0.0663
MetaTransformer	13-mer, stride=13	11	27.30%	0.0182

[†] RC TTA result (+0.78 pp over forward-only 66.29%); MetaTransformer results are forward-only.

Table 4.19: Tokenization ablation (species-level, 1,535 classes): MetaTransformer under six tokenization configurations, trained on the same 50M balanced reads.

Model	Tokenization	Tokens/read	Top-1 Acc	Macro F1
MetaTransformer	13-mer, stride=1	138	49.62%	0.4950
MetaTransformer	12-mer, stride=1	139	16.79%	0.1439
MetaTransformer	6-mer, stride=1	145	9.13%	0.0719
MetaTransformer	6-mer, stride=6	25	6.44%	0.0470
MetaTransformer	12-mer, stride=12	12	0.64%	0.0023
MetaTransformer	13-mer, stride=13	11	0.51%	0.0026

Three independent effects can be read from Tables 4.18–4.19:

Effect of switching to the NT-v2 backbone (6-mer tokenization held; depth and codebase are not). NT-Genus (66.29% forward) outperforms MetaTransformer with the same non-overlapping 6-mer tokenization (47.00%) by **+19.29 pp**. This is an operational gap, not an isolated pre-training effect: MetaTransformer uses one encoder block, NT-v2 uses 29 layers, and the two codebases differ. Section 4.8 already showed that a 16-layer from-scratch 6-mer in our codebase reaches 67.94%—above NT-v2—so MetaTransformer’s

47.00% 6-mer result understates the 6-mer ceiling. The number that isolates tokenization *inside* MetaTransformer is the next paragraph, not this +19.29 pp.

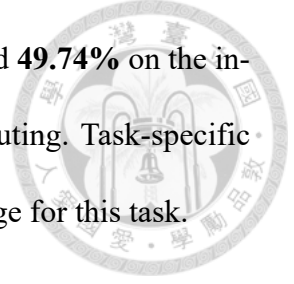


Effect of tokenization (MetaTransformer architecture held constant). Tokenization design has a far larger effect than pre-training. Two sub-effects compound:

1. **Stride (overlapping vs. non-overlapping):** Across all k values, overlapping stride=1 dominates non-overlapping stride= k . The gap widens dramatically with k : +1.87 pp for $k = 6$, +38.17 pp for $k = 12$, and +60.12 pp for $k = 13$. Non-overlapping tokenization with large k reduces each 150 bp read to as few as 11 tokens, discarding most sequence context.
2. **k -mer length (overlapping only):** Among stride=1 configurations, longer k yields monotonically higher accuracy: 6-mer (48.87%) < 12-mer (78.83%) < 13-mer (87.42%) at genus level, and 9.13% < 16.79% < 49.62% at species level. Longer tokens are taxonomically more specific—hexamers are shared across distant taxa, while 12-mers and 13-mers increasingly identify individual lineages.

Tokenization vs. NT-v2: MetaTransformer 13-mer. MetaTransformer with overlapping 13-mer tokenization, trained *from scratch* on the 50M dataset (45M training reads), achieves **87.42%** genus Top-1 accuracy—+**20.35 pp** above NT-Genus with RC TTA (67.07%), with a transformer encoder $\sim 100\times$ smaller and no pre-training (though its large- k embedding table makes its *total* parameter count far larger than NT-v2's; see the parameter-accounting caveat in Section 4.7). Against the depth-matched 16-layer 6-mer (67.94%), the best-vs-best gap is still ~ 20 pp (87.42% vs. 67.94%), even though the 13-mer model uses one encoder block and the 6-mer model uses 16. At species level, MT 13-mer

stride=1 reaches **49.62%** (1,535 classes) on the 5M validation set and **49.74%** on the independent 100K test set (Section 4.10.3), without any hierarchical routing. Task-specific tokenization is a more decisive factor than pre-trained prior knowledge for this task.



Decomposition from a common MetaTransformer 6-mer baseline. Because reverse-complement TTA is unavailable for MetaTransformer, Table 4.20 changes one factor at a time from the from-scratch MetaTransformer non-overlapping 6-mer (47.00% forward). The overlapping-12-mer and 13-mer rows hold the MetaTransformer architecture fixed and are therefore tokenization effects (+31.83 pp and +40.42 pp). The NT-v2 row does *not* hold depth or codebase fixed; it is included only as the operational gap (+19.29 pp) that Section 4.8 already showed is not pre-training. Tokenization remains the largest single-factor change on this table.

Table 4.20: Single-factor changes from a common from-scratch MetaTransformer 6-mer (non-overlapping) baseline at 50M reads (47.00% forward genus Top-1). Overlapping 12-mer and 13-mer hold the MetaTransformer architecture fixed. The NT-v2 row additionally changes depth, width, pre-training, and codebase, and is not an isolated pre-training effect (Section 4.8). All values are forward-only.

Single-factor change from MT 6-mer scratch (47.00%)	Genus Top-1 (fwd)	Δ
+ NT-v2 backbone (depth, pre-training, codebase also change)	66.29%	+19.29 pp
+ overlapping 12-mer tokenization (MT held)	78.83%	+31.83 pp
+ overlapping 13-mer tokenization (MT held)	87.42%	+40.42 pp

The bottleneck is representational: a train-fit ceiling. The decomposition above measures held-out accuracy, which conflates representational capacity with generalisation. A sharper test asks whether each model can fit its *own training data*: a model that cannot separate the training set is limited by its input representation, not by capacity, data volume, optimisation, or overfitting. Table 4.21 reports training and validation accuracy at the largest (250M-read) scale. The NT-v2 6-mer models plateau at 63–66% *on the train-*

ing set itself, essentially equal to their validation accuracy (train–val gap ≤ 1 pp)—they are not overfitting, they simply cannot fit. By contrast, MetaTransformer with overlapping 13-mer tokenisation fits the same task to 98.9% training accuracy (97.5% validation).

The implication is decisive: with non-overlapping 6-mer tokenisation, reads from different genera are mapped to token sequences too similar to be separated, so no amount of additional capacity, data, or training can raise accuracy—the ceiling is imposed before learning begins, by the tokenizer. Overlapping 13-mers preserve enough sequence detail that the *same model family* fits the data almost perfectly. This train-fit ceiling is the strongest single piece of evidence that tokenization, rather than backbone capacity or data volume, is the operative bottleneck for 6-mer GFMs on this task.

Table 4.21: Train-fit ceiling at 250M reads: the 6-mer models cannot fit the training set beyond $\sim 66\%$, whereas an overlapping 13-mer model fits it to $\sim 99\%$. Training and validation accuracy are essentially equal for the 6-mer models, ruling out overfitting and isolating the input representation as the bottleneck. NT-v2 figures are forward micro-accuracy; the MetaTransformer training figure is micro-precision.

Model (250M reads)	Train acc	Val acc	Train–val gap
NT-v2 6-mer, from scratch	63.2%	64.1%	–0.9 pp
NT-v2 6-mer, warm-start	65.8%	66.6%	–0.8 pp
MetaTransformer 13-mer (overlap)	98.9%	97.5%	+1.4 pp

Convergence and capacity ceiling (12-mer). To determine whether 78.83% is the true ceiling for the 12-mer model, training was extended into a second epoch. Train loss dropped sharply from 0.764 to 0.119, while validation loss *worsened* from 1.203 to 2.288—immediate overfitting. The 50M-read dataset saturates the 5M-parameter single-encoder MetaTransformer within one epoch.

Overlapping tokenization within pre-trained GFMs (5M scale). The MetaTransformer ablation above is conducted entirely from scratch at 50M scale. To isolate the effect of

overlapping tokenization *within the pre-trained GFM family*, we additionally fine-tune two other pre-trained DNA language models on the same 5M balanced dataset using identical training settings (LoRA $r = 16$, attention-pooling head):



- **DNABERT** [10]: BERT-base (86M params) pre-trained on the human reference genome with overlapping 6-mer tokenization (stride=1, 145 tokens per 150 bp read).
- **DNABERT-2** [11]: BERT-base (117M params) pre-trained on a multi-species genome collection using learned BPE tokenization (~ 34 tokens per read).

Table 4.22: Pre-trained GFM backbone comparison at 5M balanced reads (genus-level, 120 classes). All models use LoRA $r = 16$ fine-tuning and the same attention-pooling head. RC TTA is applied to all models.

Model	Tokenization	Tokens/read	Params	RC TTA Acc	Macro F1
NT-Genus-5M	6-mer ($s=6$, non-overlap)	25	499M	63.05%	0.449
DNABERT + LoRA	6-mer ($s=1$, overlap)	145	86M	61.78%	0.384
DNABERT-2 + LoRA	BPE	~ 34	117M	58.88%	0.323

Three observations follow from Table 4.22.

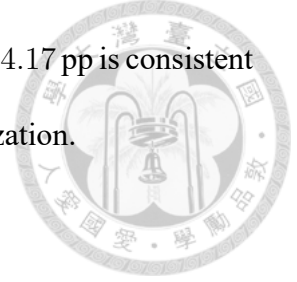
First, NT-Genus-5M with *non-overlapping* 6-mer tokenization outperforms DNABERT with *overlapping* 6-mer tokenization by +1.27 pp. At 5M training reads, the pre-training advantage from NT-v2’s larger backbone (499M vs. 86M parameters) and more diverse pre-training corpus (850 microbial genomes vs. human genome only) outweighs the tokenization benefit of overlapping stride. This contrasts with the MetaTransformer results: when starting from scratch on the same data, overlapping tokenization provides a large benefit (+1.87 pp at $k = 6$, +60.12 pp at $k = 13$); but when starting from a strong pre-trained representation, the non-overlapping NT-v2 backbone already captures sufficient local context through its positional encodings.

Second, DNABERT with overlapping 6-mer tokenization (pre-trained, 86M, 5M data, 61.78%) outperforms MetaTransformer with the same overlapping 6-mer tokenization (from scratch, one encoder block, 50M data, 48.87%), despite ten times less training data. This is an operational data-efficiency comparison at matched k ; it is not depth-matched, and it should not be read as the isolated pre-training effect already measured in Section 4.8.3.

Third, DNABERT-2’s BPE tokenization (58.88%) performs worse than either 6-mer model despite having a similar parameter count to DNABERT. BPE segments DNA by frequency-based byte pairs rather than fixed-length k -mers, which may fragment taxonomically informative motifs; the learned vocabulary may not align well with the k -mer statistics that distinguish metagenomic reads.

Methodological note on training equivalence. The three models use the same learning rate (5×10^{-4} head/LoRA, 3×10^{-5} backbone), warmup ratio (5%), cosine decay schedule, and epoch count (30), but differ in batch size (NT-v2: 32, DNABERT: 128, DNABERT-2: 256) due to GPU memory constraints imposed by sequence length differences (25, 145, and ~ 34 tokens per read respectively). Consequently, the total number of gradient updates differs: approximately 453K for NT-v2, 117K for DNABERT, and 59K for DNABERT-2. All three learning curves were still improving at epoch 30 (gain < 0.1 pp per epoch), with the cosine schedule decaying the learning rate to near zero by the final epoch. The comparison is therefore epoch-based—each model has seen the training data 30 times—rather than step-based or compute-matched. This is standard practice in fine-tuning comparisons, and the observed ranking (NT-v2 $>$ DNABERT $>$ DNABERT-2) is unlikely to be reversed by additional training: DNABERT, despite receiving $3.9\times$ fewer gradient

updates than NT-v2, lags by only 1.27 pp, while DNABERT-2’s gap of 4.17 pp is consistent across all training stages and attributable primarily to its BPE tokenization.



Sample-level evaluation of the three GFM backbones (5M scale). Read-level accuracy alone does not capture how a backbone choice propagates to the sample-level abundance and detection metrics that matter in practice. We therefore apply the sample-level evaluation protocol (5M validation pool, 50K reads per sample, 100 partition samples; Section 4.11) to the RC-TTA predictions of all three pre-trained backbones at 5M scale.

Table 4.23 reports the results.

Table 4.23: Sample-level genus evaluation of the three pre-trained GFM backbones at 5M balanced reads (RC TTA predictions, 5M validation pool, 50K reads/sample).

Model	Read Acc	Pearson r	Bray-Curtis	ROC AUC	Sens @ 95% Spec
NT-Genus-5M	63.05%	0.9928	0.112	0.666	14.5%
DNABERT + LoRA	61.78%	0.9906	0.113	0.672	14.2%
DNABERT-2 + LoRA	58.88%	0.9915	0.141	0.640	11.6%

Two patterns emerge. First, abundance estimation (Pearson r) saturates at ~ 0.99 for all three backbones, consistent with the genus-level finding that relative-abundance correlation is forgiving at low task granularity (Section 4.11.2)—the read-accuracy ranking (NT-Genus > DNABERT > DNABERT-2) does not separate the backbones on Pearson r . Second, the discriminating metrics are Bray-Curtis dissimilarity and binary detection: DNABERT-2 is clearly worst on both (Bray-Curtis 0.141 vs. 0.112–0.113; ROC AUC 0.640 vs. 0.666–0.672; sensitivity at 95% specificity 11.6% vs. 14.2–14.5%), mirroring its lower read accuracy and reinforcing that BPE tokenisation is the weakest of the three for this task. NT-Genus and DNABERT are nearly indistinguishable at the sample level, with NT-Genus marginally ahead on Bray-Curtis and sensitivity and DNABERT marginally ahead on ROC AUC.

Same-architecture tokenizer comparison (16 layers). The MetaTransformer k /stride grid holds one codebase fixed. Table 4.24 holds *our* 16-layer encoder, the 50M gut-read pool, and `clean_common` RC TTA fixed, and varies only the tokenizer (Section 3.2.5).

All three Taiwan-2 arms (Jobs A/B/C) trained the full 15 epochs.

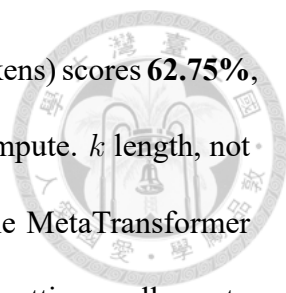
Table 4.24: Same-architecture tokenizer comparison on 50M gut reads, scored on `clean_common` 100K with RC TTA. The 16-layer non-overlapping 6-mer and 29-layer 6-mer rows are local runs; Jobs A/B/C were trained on Taiwan-2. MetaTransformer 13-mer is the one-block published-protocol number from Table 4.18, not a 16-layer run.

Model	Tokenizer	Params	Tok/read	Top-1	Bal. acc
16L from scratch	overlap 6-mer (Job B)	3.88M	146	62.75%	0.353
16L from scratch	non-overlap 6-mer	3.88M	25	67.94%	0.435
29L from scratch	non-overlap 6-mer	6.45M	25	69.52%	0.462
NT-v2 + LoRA	non-overlap 6-mer	500M	25	67.08%	0.398
16L from scratch	hashed 13-mer, 2 GiB (Job C)	540M	139	83.83%	0.669
16L from scratch	exact 13-mer, 8.6 GiB (Job A)	2.15B	139	85.63%	0.732
MetaTransformer (1 block)	exact 13-mer	—	139	87.42%	—

Four results follow.

(i) **Long k works at matched depth.** Hashed 13-mer reaches **83.83%**—+15.89 pp over the 16-layer non-overlapping 6-mer and +16.75 pp over NT-v2—with a 2 GiB embedding rather than the 48 GiB a dense 4^{13} table would need. Hashing costs **1.80 pp** versus the exact table (85.63%). Macro F1 rises faster than Top-1: overlapping 6-mer Job B has macro F1 0.385, hashed 13-mer 0.732 ($1.90\times$), exact 13-mer 0.747. The 6-mer long tail, not just Top-1, is the tokenizer.

(ii) **Depth compensates for a weak tokenizer.** With a 6-mer tokenizer, 1 $\rightarrow$ 29 layers is worth +15.60 pp (53.92% $\rightarrow$ 69.52%). With a 13-mer tokenizer, MetaTransformer reaches 87.42% with one encoder block while our 16-layer exact 13-mer reaches 85.63%. Depth and pre-training were substituting for a representation that discards the signal. A one-layer hashed 13-mer in our codebase was not run; MetaTransformer’s 87.42% protocol is the 50M matched run in Table 4.18, not an independent re-implementation.



(iii) **Overlapping 6-mer is harmful.** Overlapping 6-mer (146 tokens) scores **62.75%**, -5.19 pp against non-overlapping 6-mer (25 tokens) at far higher compute. k length, not overlap at $k = 6$, is what matters here. This does not contradict the MetaTransformer result that overlap is essential at $k = 12/13$: those non-overlapping settings collapse to 11–12 tokens per read.

(iv) **Caveats.** Job A differs from B/C in exact vs. hashed vocabulary, d_{model} 64 vs. 128 (memory-forced), and a canonical (RC-invariant) vocabulary. The width confound favours C, so the exact-vocabulary advantage is not a width artefact. Job A ran fp16; B and C ran bf16. Only `evaluate.py --rc_tta on clean_common 100K` is comparable across arms. Canonical-vocabulary analysis predicted RC-TTA behaviour before the numbers existed: Job A $+0.01$ (TTA is a no-op), Job C $+6.71$ (non-canonical hashing), Job B -0.03 (rc_augment already taught invariance).

Implication for NT-v2. NT-v2 is constrained to 6-mer tokenization by its pre-training design. Switching to overlapping 13-mer tokenization on the same 50M reads yields $+15.89$ pp (hashed, 16-layer) to $+20.35$ pp (MetaTransformer) without using NT-v2 pre-training. A genomic foundation model pre-trained with overlapping long- k tokenization on microbial genomes would combine that tokenizer with pre-training as a data-efficiency mechanism (Section 4.8.3). General-purpose GFM of the NT-v2/DNABERT family do not currently provide that combination; METAGENE-1 and GenomeOcean use BPE and were not tested (Chapter 5).



4.10 Species-Level Classification at Scale

Genus accuracy saturates near 67% under 6-mer tokenization (Section 4.4). Species classification (1,535 classes) is the stricter test of whether the same representation can resolve intra-genus variation. A 500K pilot reached only 8.32% direct Top-1 and 20.88% under oracle-genus masking—far above chance (0.065%) but too data-starved to interpret—so the remainder of this section uses the 50M scale.

4.10.1 NT-Species at 50M

The species model is initialized from the NT-Genus backbone (`--init_from`), transferring LoRA weights and replacing the 120-way head with a 1,535-way head. Training converged at epoch 30 (Figure 4.2, right panel) to **17.55%** forward Top-1 on the 5M held-out set, with Top-3/5/10 of 31.26%/38.19%/47.65%, balanced accuracy 17.56%, macro F1 0.157, and 191/1,535 species at F1 = 0 (12.4%). The same learning-rate restart that affected NT-Genus appeared at epoch 7 (−0.79 pp) and was corrected from epoch 13. Full-scale RC TTA and routing on all 5M reads exceeded the 200 GB memory ceiling of the evaluation script (it materialises $5M \times 1,535$ logit arrays per mode); hierarchical comparisons therefore use the independent 100K set below. The genus-level RC TTA gain is already measured at +0.78 pp (Section 4.4.2).

4.10.2 Independent Test Set and Data-Leakage Audit

Hierarchical modes restrict the candidate set, so even modest train–test overlap can inflate oracle and per-genus scores. The 100K subset first used for these modes had been sampled from the training FASTA *before* the train/validation split and shared >99% of

reads with the 45M training pool. We replaced it with 100K reads from MetaTransformer’s `val_reads.fa`—the same ART protocol and 2,505 HGR-UMGS genomes, 0% overlap with our training stream. Re-scoring NT-Genus gave 66.6% on the overlapping 100K versus 66.1% on the independent 100K (0.5 pp); the NT-Species flat baseline moved by less than 0.4 pp. That is evidence against read-level memorisation, but all hierarchical numbers below use the independent set. Genus results in Section 4.4.2 use the standard 5M validation split and are unaffected.

4.10.3 Hierarchical Evaluation

Table 4.25 reports every routing mode on the independent 100K set. NT-Genus is 66.1% on this set. The MetaTransformer hierarchical rows use one genus model plus per-genus logit masking, not a per-genus ensemble.

Table 4.25: Species Top-1 on the independent 100K-read test set (1,535 classes). Per-genus (50M) is 81 dedicated NT-v2 heads with a frozen backbone. MT hierarchical uses a single genus router and species-logit masking.

Model	Mode	Top-1	F1 (macro)
NT-Species (ep30)	Flat	15.9%	0.135
	Top-5 / soft genus routing	15.9% / 15.8%	0.136 / 0.135
	Oracle (true genus)	27.4%	0.252
Per-genus (50M)	Predicted genus (NT-Genus)	15.6%	0.152
	Oracle genus	29.5%	0.283
MT 13-mer ($s=1$)	Flat / hierarchical (87.5% genus)	49.74% / 50.86%	—
MT 6-mer ($s=1$)	Flat / hierarchical (48.9% genus)	9.22% / 6.41%	—

Four conclusions follow, and they are all that the extra routing experiments add.

Per-genus specialisation is real under a perfect router. Oracle-genus routing lifts the 81 per-genus 50M classifiers to 29.5% Top-1 (macro F1 0.283), above the flat NT-Species oracle (27.4%, F1 0.252). The specialists use a *frozen* backbone, so the gain is

from decomposing the 1,535-way problem, not from extra fine-tuning.

Predicted routing does not realise that gain. With NT-Genus (66.1%), the per-genus pipeline is 15.6%, 0.3 pp *below* the flat 15.9% baseline. A wrong genus sends the read to a candidate set that cannot contain the true species. Flat top-5 / soft routing moves Top-1 by at most 0.1 pp. The binding constraint is genus accuracy, not the species head.

Tokenization sets a higher ceiling than routing can recover. MT overlapping 13-mer, with no hierarchical routing, reaches **49.74%** on the same set—+20.2 pp above the best NT-v2 number under any routing mode (29.5%). The 6-mer intra-genus ceiling sits in the high-20s; long- k tokenization more than doubles it.

Masking helps only when the router is already strong. MT 13-mer genus accuracy is 87.5%, and masking adds +1.12 pp (49.74% → 50.86%). MT 6-mer genus accuracy is 48.9%, and the same masking *costs* 2.81 pp (9.22% → 6.41%). NT-v2 sits in between: oracle +11.5 pp on the flat model, predicted routing −0.3 pp on the specialists. Hierarchical masking is beneficial if and only if it removes more wrong candidates than it accidentally excludes.

The 81 multi-species heads were trained on genus-scoped 50M subsets (~32,600 reads per species) with the backbone frozen; 39 singleton genera are assigned deterministically. Mean per-genus validation accuracy is 48.3% (median 44.2%), but four large genera dominate the error: *Collinsella* (349 species, 1.3%), *Clostridium* (191, 19.9%), *Prevotella* (91, 26.6%), and *Bacteroides* (79, 15.9%). Genera with ≤ 5 species average >80%; those with >50 species average <20%.

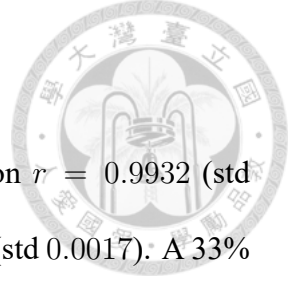
4.10.4 Subgenus Routing: A Negative Result

We tested whether a K -means partition of NT-v2 embeddings (~ 25 – 30 species per subgroup, including *Blautia*) could shrink those large genera. For *Collinsella* the silhouette coefficient is 0.03; a UMAP projection is an undifferentiated blob. Hard and soft subgenus routing both *lower* species Top-1 relative to the per-genus baseline they were compared against (predicted 14.7% $\rightarrow$ 13.2%/13.6%; oracle 27.8% $\rightarrow$ 25.6%/26.1%). The finalised per-genus numbers in Table 4.25 are 15.6%/29.5%; the negative conclusion is unchanged. Adding a routing tier without a representation that separates species is counterproductive. The bottleneck is 6-mer within-genus resolution, not hierarchy depth.

4.11 Sample-Level Application Evaluation

Per-read accuracy is not the quantity most metagenomic studies report. This section asks whether the 67.07% NT-Genus model, and the species models above, produce usable relative abundances and presence/absence calls. Metrics follow CAMI and Bracken practice [41]–[44]: Pearson r and Spearman ρ for abundance, Bray–Curtis dissimilarity for magnitude-aware error, and ROC AUC for detection. All genus numbers reuse the NT-Genus RC TTA predictions of Section 4.4.2; no extra GPU inference is required.

Two synthetic constructions are taken from the 5M validation pool (120 genera). **Random-partition** samples ($100 \times 50,000$ reads) keep every genus present; because training is species-balanced, the expected abundance of genus g is $n_g/1,535$. **Sparse-community** samples ($200 \times 50,000$ reads) include exactly 60 of 120 genera, supplying true negatives for detection.



4.11.1 Genus Abundance and Detection

On the 100 random-partition samples, NT-Genus yields Pearson $r = 0.9932$ (std 0.0004), Spearman $\rho = 0.9312$ (std 0.0080), and Bray–Curtis 0.0937 (std 0.0017). A 33% per-read error rate still aggregates, because misclassifications spread across genera and largely cancel. High-abundance genera are slightly overestimated (classifier bias toward genera with more member species), not because of genome-length artefacts in the ART schedule (Section 3.5).

That $r = 0.993$ is *not* a test of variable communities. Every partition sample shares the same expected abundance profile; the correlation measures stability under binomial sampling noise. Sparse-community detection is the stricter probe. With a ≥ 1 -read call, sensitivity is 93.9% at 0.01–0.1% true abundance, 97.6% at 0.1–1%, and 100% at $\geq 1\%$. ROC AUC with predicted abundance as the score is 0.705; sensitivity is 17.2%/30.9%/46.2% at 95%/90%/80% specificity. The modest AUC is a noise floor: at 33% error, an absent genus in a 50,000-read sample is expected to receive $\sim 50,000 \times 0.33/120 \approx 138$ spurious reads ($\sim 0.28\%$). A 95%-specificity threshold must sit at 1.49% predicted abundance and therefore also suppresses genuinely rare taxa. Abundance estimation is forgiving at 120 classes; calibrated presence/absence is not.

4.11.2 Matched Protocol across Models

Table 4.26 repeats the same sample protocol for NT-Genus and two MetaTransformer checkpoints trained on the same 50M reads.

Relative abundance at genus rank barely separates the methods ($r = 0.984$ – 0.999). Detection does: MT 13-mer AUC 0.900 versus 0.705/0.569 for the two 6-mer models,

Table 4.26: Genus sample-level evaluation under one protocol (100 partition samples, 200 sparse samples, 50K reads). Read accuracies are 67.07% (NT-Genus, RC TTA), 48.82% (MT 6-mer $s=1$), and 87.42% (MT 13-mer $s=1$).

Metric	NT-Genus	MT 6-mer	MT 13-mer
Pearson r (abundance)	0.993	0.984	0.999
Bray–Curtis	0.094	0.167	0.028
Sensitivity $\geq 1\%$	100%	99.9%	100%
Sensitivity 0.1–1%	97.6%	86.8%	100%
ROC AUC	0.705	0.569	0.900
Sensitivity @ 95% spec	17.2%	9.4%	47.7%

matching the noise-floor arithmetic (expected spurious reads fall from ~ 138 at 67% accuracy to ~ 54 at 87%). NT-Genus beats the from-scratch 6-mer MetaTransformer on every sample metric; the two models also differ in depth, width, and codebase, so this is an operational comparison at matched k , not an isolated pre-training effect (Section 4.8).

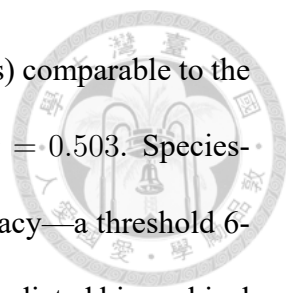
4.11.3 Species-Level Samples and Kraken2

The independent 100K set has only ~ 65 reads per species, so samples are smaller (1,000 reads; 100 partitions; 50 genera present in sparse samples). Table 4.27 keeps the rows that change the conclusion.

Table 4.27: Species-level sample evaluation on the independent 100K set (1,535 classes, 1K reads/sample). Kraken 2 uses a custom $k=35$ database from the same 2,505 UMGS+HGR genomes; unclassified reads (30%) count in the denominator but add zero predicted abundance. The MT 13-mer hierarchical species checkpoint did not converge (val_loss = 9.29 at batch 10,000) and is omitted here; its read-level numbers in Table 4.25 remain valid.

Model	Read Acc	Pearson r	Bray–Curtis	ROC AUC
NT-Species flat	17.8%	0.135	0.589	0.794
NT-Species hierarchical	15.8%	0.106	0.608	0.796
Per-genus (predicted / oracle)	15.6% / 29.5%	0.083 / 0.159	0.625 / 0.541	0.789 / 0.842
MT 6-mer flat / hierarchical	9.0% / 6.4%	0.066 / 0.041	0.652 / 0.682	0.692 / 0.639
MT 13-mer flat	53.7%	0.503	0.353	0.973
Kraken 2 (in-DB)	66.2%	0.727	0.218	0.925

At 1,535 classes a 1,000-read sample gives ~ 0.65 true reads per species. NT-Species



at 17.8% therefore has a noise floor (~ 0.54 spurious reads per species) comparable to the signal, and Pearson r stalls at 0.135. MT 13-mer at 53.7% reaches $r = 0.503$. Species-level abundance becomes useful only around 40–50% per-read accuracy—a threshold 6-mer models in this thesis never cross and overlapping 13-mer does. Predicted hierarchical masking again hurts when the router is weak (NT-Genus 66.1%: r 0.135 $\rightarrow$ 0.106; MT 6-mer 48.9%: 0.066 $\rightarrow$ 0.041). Oracle per-genus routing is the NT-v2 ceiling (29.5%, $r = 0.159$) and remains 24.2 pp of read accuracy behind MT 13-mer flat.

Kraken 2 on the same references reaches 66.2% species Top-1 and $r = 0.727$, committing to 70% of reads (94.6% species / 99.3% genus among those). The remaining 30% abstain. Restricting to the 85,819 reads whose species are in the Kraken 2 FASTA collection (219 of 1,535 species had no local assembly) raises Kraken 2 to 77.18% species Top-1 versus 52.64% for MT 13-mer flat. At *genus* rank the comparison flips for uncorrected Kraken 2 counts (NT-Genus $r = 0.992$, MT 13-mer $r = 0.9993$, raw Kraken 2 $r = 0.823$ on that restricted pool), but Bracken [43] restores Kraken 2+Bracken to $r = 0.997$. Neural models therefore have *no* sample-level abundance advantage against a standard Kraken 2+Bracken pipeline on these simulated communities. What remains is (i) read-level accuracy under a suitable tokenizer and (ii) the fact that a neural classifier trains from labelled reads and does not need a per-species reference FASTA—Kraken 2 accuracy on the 14,181 uncovered GCF reads is 0% by construction, whereas those species are in-distribution for every neural model here.

The practical summary is short. Genus relative abundance on synthetic, near-balanced partitions cannot choose a method. Species abundance requires a tokenizer that already clears $\sim 50\%$ read accuracy. Hierarchical experts and subgenus routing do not lift a 6-mer GFM above that line. Kraken 2+Bracken matches neural genus abundances on in-

database simulated reads; learning-based methods earn their keep only where assemblies are missing or the tokenizer is long- k .

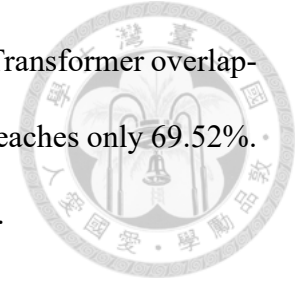


4.12 Summary of Findings

The experimental results lead to several key conclusions:

1. **Inside a fixed 6-mer tokenizer, data volume dominates tricks:** A $10\times$ data increase yields +7.76 pp, while training-technique optimizations yield at most ± 0.5 pp. Beyond 50M the 6-mer model saturates (+0.22 pp at 250M); an overlapping 13-mer does not.
2. **RC TTA is beneficial for pre-trained backbones:** Reverse-complement test-time augmentation consistently improves accuracy by +0.1–1.5 pp, with near-zero gain for randomly initialized models.
3. **Class balance matters:** Balanced sampling across species eliminates the “dead class” problem ($F1=0$ classes: $9 \rightarrow 2$) and improves macro F1 (+12.27 pp).
4. **Overfitting signals data starvation at 500K:** The train-validation gap shrinks from 6.25% at 500K to 1.30% at 5M.
5. **The +13.19 pp one-layer gap is depth, not pre-training:** At matched 6-mer tokenization on `clean_common`, 1 / 8 / 16 from-scratch layers score 53.92% / 65.44% / 67.94% versus NT-v2 67.08%. Depth accounts for +14.02 pp; pre-training at 50M is ≈ 0 . At 5M the same 16-layer match gives NT-v2 +8.94 pp (63.05% vs. 54.11%). Pre-training buys data efficiency, not a higher 6-mer ceiling.
6. **Tokenization is the bottleneck, including at matched architecture:** Hashed 13-

mer 16-layer reaches 83.83% and exact 13-mer 85.63%; MetaTransformer overlapping 13-mer reaches 87.42%. A 29-layer from-scratch 6-mer reaches only 69.52%. Overlapping 6-mer is harmful (−5.19 pp vs. non-overlapping).



7. **Hardware is sufficient:** Current NVIDIA H100 hardware is adequately provisioned; the operational bottleneck is the SLURM time limit rather than computational resources.
8. **Species hierarchy cannot outrun the tokenizer:** Per-genus specialists beat a flat oracle only when the genus is given (29.5% vs. 27.4%); predicted routing at 66.1% genus accuracy does not (15.6% vs. 15.9%). Subgenus *K*-means on NT-v2 embeddings is a negative result. Overlapping 13-mer reaches 49.74% flat—+20.2 pp above the best NT-v2 routing—and masking helps only when the genus router is already strong (Section 4.10.3).
9. **Genus abundance cannot choose a method; species abundance can:** On synthetic partitions, Pearson r is 0.984–0.999 across 6-mer and 13-mer models. Species-level r stays near 0.14 until per-read accuracy crosses ~40–50%. Kraken2+Bracken matches neural genus abundances ($r = 0.997$); Kraken2 wins on in-database simulated species reads and is undefined on the 219 species without a local FASTA (Section 4.11).



Chapter 5

Conclusion and Future Work

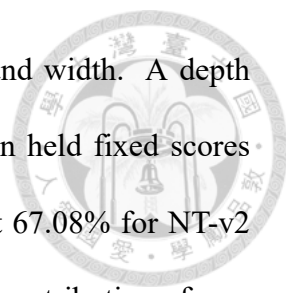


5.1 Summary of Contributions

This thesis systematically evaluates the suitability of general-purpose genomic foundation models (GFMs) for metagenomic short-read taxonomic classification. Through controlled ablations of data scale, encoder depth, pre-training, and tokenization, we identify which design factors are decisive and which are not. Our main findings are:

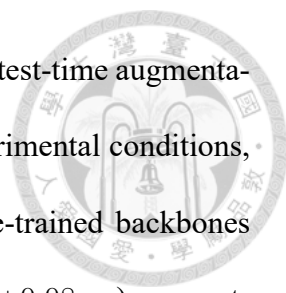
Tokenization sets the ceiling; data volume dominates only inside a fixed tokenizer. Through systematic data scaling spanning 500K to 50M training reads, we show that within the NT-v2 6-mer framework, increasing training data by $10\times$ ($500\text{K} \rightarrow 5\text{M}$) yields +7.76 pp in accuracy and +12.27 pp in macro F1; a further $10\times$ ($5\text{M} \rightarrow 50\text{M}$) adds +4.02 pp. Pushing a further $5\times$ to 250M reads adds only +0.22 pp (67.29%), empirically refuting the log-linear projection: the NT-v2 6-mer model saturates beyond 50M. This saturation is tokenizer-specific rather than a data limit: trained on the same reads over the same $50\text{M} \rightarrow 250\text{M}$ range, an overlapping 13-mer model scales from 87.5% to 98.7% genus Top-1. By contrast, architectural and training-technique ablations on 500K reads yield at most ± 0.5 pp. The token-level pipeline—attention-pooling head fine-tuned with LoRA on the NT-v2 backbone—reaches 67.07% genus Top-1 RC TTA at 50M reads, compared to 51.70% for the frozen mean-pool baseline.

The +13.19 pp one-layer gap is depth, not pre-training; pre-training buys data efficiency. A 29-layer pre-trained NT-v2 versus a one-layer randomly initialized Transformer at the same 50M scale and 6-mer tokenizer differs by +13.19 pp RC TTA (67.07%



vs. 53.88%). That comparison confounds pre-training with depth and width. A depth sweep with the tokenizer, data, head, and `clean_common` evaluation held fixed scores 53.92% / 65.44% / 67.94% at 1 / 8 / 16 from-scratch layers, against 67.08% for NT-v2 LoRA: depth accounts for +14.02 pp (1 $\rightarrow$ 16), and at 50M the net contribution of pre-training is $\approx$ 0 (the 3.88M-parameter 16-layer model is +0.86 pp higher). The 8- and 16-layer models are trained 100% from scratch, so the $\sim$ 68% 6-mer ceiling is not an artefact of LoRA updating only $\sim$ 1% of NT-v2. The ranking reverses at 5M reads under the same 16-layer match: NT-v2 63.05% versus from-scratch 54.11% (+8.94 pp). Pre-training therefore helps when labels are scarce; it does not raise the 6-mer ceiling once 50M labelled reads are available. Within the family of pre-trained GFMs at 5M reads, NT-v2 still outperforms DNABERT by +1.27 pp and DNABERT-2 by +4.17 pp.

Tokenization remains the dominant design factor, including at matched architecture. Holding MetaTransformer fixed and varying $k \in \{6, 12, 13\}$ and stride, overlapping stride=1 is critical (up to +60 pp over non-overlapping at $k = 13$), and longer k yields monotonically higher accuracy among overlapping configurations. MetaTransformer with overlapping 13-mer tokenization, trained from scratch on the same 50M reads, achieves **87.42%** genus Top-1—+**20.35 pp** above NT-v2 + LoRA (67.07%). A same-architecture 16-layer comparison on `clean_common` RC TTA reaches 83.83% with a hashed 13-mer (2 GiB embedding) and 85.63% with an exact 13-mer, versus 67.94% for the 16-layer non-overlapping 6-mer; a 29-layer from-scratch 6-mer reaches only 69.52%. Even when the 6-mer model is given 16 layers and the 13-mer MetaTransformer only one, the 13-mer still wins by $\sim$ 20 pp (87.42% vs. 67.94%). Depth compensates for a weak tokenizer; it does not break the 6-mer ceiling. MetaTransformer’s published advantage on this task is therefore its 13-mer tokenizer, not a uniquely strong encoder.



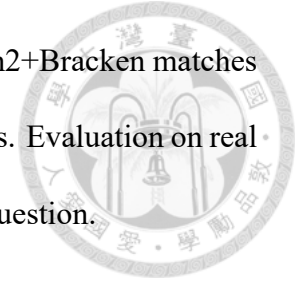
RC TTA is a robust, free improvement. Reverse-complement test-time augmentation consistently improves accuracy by +0.1 to +1.5 pp across experimental conditions, requiring no additional training cost. The benefit is largest for pre-trained backbones (+0.78 to +1.54 pp) and near-zero for randomly initialized models (+0.08 pp), supporting the interpretation that RC TTA primarily corrects pre-training-induced strand biases rather than stochastic inference noise.

Class balance is critical for rare taxa. Balanced subsampling across species reduces the number of F1=0 classes from 9 to 2 and improves macro F1 by +12.27 pp, demonstrating that class imbalance—not model capacity—is the primary cause of poor performance on rare taxa at the 500K–5M scales.

Per-genus specialisation validates the hierarchical architecture; subgenus routing does not. Evaluation on a 100K independent test set demonstrates that per-genus 50M LoRA classifiers under oracle-genus routing reach 29.5% species Top-1, exceeding the flat sp_v4 oracle ceiling of 27.4%. Predicted routing through the 66.1%–67.07% genus model lowers Top-1 to 15.6%, marginally below the flat baseline (15.9%), because per-genus specialists do not recognise reads from genera they were not trained on. Subgenus *K*-means routing on NT-v2 embeddings is a negative result (oracle Top-1 25.6–26.1%).

Per-read accuracy translates to sample-level utility under controlled conditions, but genus abundance cannot choose a method. On synthetic species-balanced random-partition samples, 67% per-read accuracy yields Pearson $r = 0.993$ for relative abundance and Bray-Curtis dissimilarity 0.094; 100% detection sensitivity is achieved for taxa at $\geq 1\%$ relative abundance. All 100 random-partition samples share the same expected per-genus abundance profile, so this result measures stability under binomial sampling noise

rather than generalisation to genuinely variable communities. Kraken2+Bracken matches neural genus abundances ($r = 0.997$) on these simulated communities. Evaluation on real and compositionally diverse microbiome samples remains an open question.

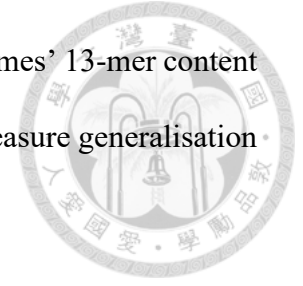


5.2 Limitations

This work has several limitations that should be acknowledged:

1. **Simulated data only:** All experiments use reads simulated by ART from reference genomes. Real metagenomic reads contain additional noise (host contamination, chimeric reads, strain-level variation) that may degrade classifier performance.
2. **Limited taxonomic scope:** The reference database covers 120 genera and 1,535 species of human gut microorganisms. Performance on environmental samples with broader taxonomic diversity is unknown.
3. **Single read length:** All experiments use 150 bp reads. Performance at other read lengths (e.g., 250 bp for Illumina MiSeq, or long reads from Oxford Nanopore / PacBio) has not been evaluated.
4. **Data scaling saturates for the 6-mer model, not universally:** We trained balanced subsets up to 250M reads (Section 4.4); the warm-started 250M NT-v2 model reaches only 67.29% RC TTA (+0.22 pp over the 50M 67.07%), confirming that the 6-mer model saturates rather than following the log-linear projection. This is a limit of the tokenizer, not the data: trained on the same reads over the same range, an overlapping 13-mer model scales from 87.5% to 98.7% genus Top-1. The gap to overlapping- k -mer methods is therefore not closable by additional training data within the NT-v2 6-mer framework. We note that the 13-mer model's 98.7% reflects

near-complete closed-set coverage of the 1,535 reference genomes’ 13-mer content (approaching a lookup over known genomes), and does not measure generalisation to novel genomes.



5. **Memory-bound loader in the primary pipeline:** The single-GPU TWCC pipeline loads training reads into memory; the 250M-scale experiments above were instead run on a multi-GPU H200 cluster with a lazy, index-based loader. Unifying the two (a streaming loader in the main codebase) remains future engineering work.
6. **Architectural capacity ceiling of MetaTransformer:** MetaTransformer’s single-encoder-block design ($\sim 5\text{M}$ encoder parameters, excluding the embedding table) saturates within one training epoch on 50M reads; a second epoch produces overfitting rather than further generalisation. A larger model would likely improve beyond 78.83% at 12-mer but was not evaluated due to infrastructure constraints.
7. **Scope is general-purpose GFMs:** We evaluated NT-v2 and, at 5M scale, DNABERT and DNABERT-2—models pre-trained for eukaryotic or mixed-genome regulatory genomics, not for short-read metagenomic taxonomy. Two GFMs pre-trained on metagenomic sequence were not tested: METAGENE-1 [27] ($\sim 1.5\text{ Tbp}$ wastewater metagenomes, BPE) and GenomeOcean [28] ($\sim 600\text{ Gbp}$ metagenomic assemblies, 4B parameters, BPE). The domain and context-length mismatch that plausibly explains NT-v2’s ≈ 0 transfer at 50M would not apply to those models in the same way, and their BPE tokenizers would introduce a further variable. Claims in this thesis are therefore scoped to general-purpose GFMs, not to all GFMs.
8. **Remaining confounds in the tokenizer comparison:** The same-architecture 16-layer 13-mer arms differ in exact versus hashed vocabulary, embedding width (d_{model}

64 vs. 128), and canonical versus non-canonical vocabulary; Job A ran fp16 and Jobs B/C ran bf16. MetaTransformer’s 87.42% is quoted from the matched 50M protocol used in Chapter 4; a one-layer hashed 13-mer in our own codebase was not run. Soil depth-matched from-scratch controls were not completed.

9. **LoRA versus full fine-tune:** NT-v2 is adapted with LoRA on $\sim 1\%$ of parameters.

A full fine-tune might score higher. The from-scratch 8- and 16-layer models, which train 100% of parameters and hit the same $\sim 68\%$ 6-mer ceiling, are the evidence that the ceiling is not solely a LoRA artefact.

5.3 Future Work

The findings above point to four primary future directions:

5.3.1 Microbial Foundation Models with Overlapping Long- k Tokenization

The tokenization ablation (Section 4.9) identifies overlapping long- k tokenization as the dominant performance driver. No publicly available GFM that we evaluated combines this property with large-scale pre-training on diverse microbial genomes; METAGENE-1 and GenomeOcean use BPE rather than long k -mers and were not tested here. The most promising direction consistent with the measurements in this thesis is therefore to pre-train a transformer on the GTDB database [45] ($>400,000$ microbial genomes) with overlapping 12-mer or 13-mer tokenization—or to evaluate hashed long- k embeddings, which reached 83.83% at 16 layers with a 2 GiB table. Open empirical questions include (i) whether RC TTA provides an equivalent benefit ($\sim +1$ pp) at $k = 12/13$, (ii) whether $k > 13$ continues to improve accuracy, (iii) whether a one-layer hashed 13-mer in our

codebase matches MetaTransformer’s 87.42%, and (iv) whether BPE models pre-trained on metagenomes (METAGENE-1, GenomeOcean) close the 50M gap that NT-v2 does not. A complementary, lower-cost direction is to fine-tune NT-v2 with a learned 12-mer projection layer; this is bounded above by the model’s pre-trained 6-mer positional encodings and is therefore expected to be a partial mitigation rather than a full solution.

5.3.2 Streaming Infrastructure for Species-Level Scaling

The genus-level data-scaling question is now settled: scaling balanced reads from 50M to 250M yields only +0.22 pp (Section 4.4), so additional *genus* training data is not a productive direction within the 6-mer framework—the earlier log-linear projection into the 82–90% range is empirically refuted (an overlapping 13-mer tokenizer, by contrast, keeps improving on the same data; Section 4.4). The streaming/DDP infrastructure built for the 250M runs nonetheless remains valuable for the *species*-level task (1,535 classes), where per-species 50M and full-scale species training are still bottlenecked by the memory-bound primary pipeline. Folding the lazy, index-based loader and multi-GPU orchestration used here into the main codebase would enable those species-level experiments without re-introducing the in-memory ceiling.

5.3.3 Hierarchical Routing Improvements for Species Classification

Per-genus 50M classifiers under oracle-genus routing reach 29.5% species Top-1 (Section 4.10.3), exceeding the flat sp_v4 oracle (27.4%) and validating per-genus specialisation. Predicted routing through the 66%–67% genus model, however, lowers Top-1 to 15.6%—marginally below the flat baseline (15.9%)—because per-genus specialists do

not recognise reads from genera they were not trained on.

A subgenus routing tier was investigated as a direct approach to reducing intra-classifier confusion for the five largest genera (Section 4.10.4). Species-level K -means partitioning on NT-v2 embeddings was used to define subgroups of ~ 25 – 30 species. Silhouette analysis (Collinsella: 0.03) and UMAP visualisation confirm that NT-v2’s 6-mer representations do not separate species within a genus, and both hard and soft subgenus routing decreased oracle Top-1 to 25.6–26.1% (below the per-genus baseline), indicating that adding a routing tier without better representations is counterproductive.

The primary path to improving hierarchical species accuracy is therefore to improve the quality of the genus router itself. Since NT-v2’s 6-mer tokenisation constrains genus accuracy to $\sim 67\%$, this is best addressed through the long- k pre-training direction above: a microbial foundation model with overlapping 13-mer tokenisation would simultaneously increase genus routing accuracy and provide richer intra-genus representations that could support a meaningful subgenus partition.

5.3.4 Real-World Validation and Calibrated Sample-Level Estimation

Validation on real metagenomic data from public repositories (e.g., the Human Microbiome Project [46]) is essential for assessing practical applicability. This includes (i) robustness to sequencing errors and platform-specific quality profiles, (ii) handling host DNA contamination, and (iii) out-of-distribution detection for taxa absent from the training database. In parallel, the random-partition abundance evaluation in Section 4.11 measures stability under fixed expected composition rather than generalisation across compo-

sitionally variable communities; future evaluation should therefore use sparse, asymmetric, and real microbiome-derived sample compositions, paired with (iv) calibrated statistical tests for presence/absence above the per-sample noise floor and (v) learned abundance correction that compensates for the observed systematic overestimation of high-abundance genera (Section 4.11.1).

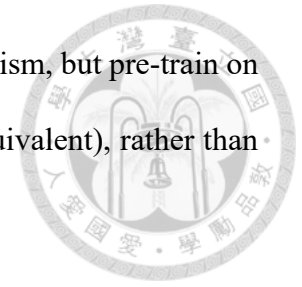
5.4 Concluding Remarks

This thesis demonstrates both the promise and the current limitations of applying general-purpose genomic foundation models to metagenomic taxonomic classification. The controlled ablation experiments yield two concrete findings that extend beyond the specific system studied here.

First, tokenization—not pre-training, depth, or further 6-mer data—sets the closed-set genus ceiling on 150 bp reads. Four architecturally different non-overlapping 6-mer models cap at 65–70% (8-layer from scratch 65.44%, 16-layer 67.94%, NT-v2 LoRA 67.08%, 29-layer from scratch 69.52%), while overlapping 13-mer models on the same 50M reads reach 83.83% (hashed, 16-layer), 85.63% (exact, 16-layer), and 87.42% (Meta-Transformer, one encoder block). The previously quoted +13.19 pp and +19.29 pp “pre-training” gaps compared a deep pre-trained model with a one-layer from-scratch model and do not survive a depth-matched control.

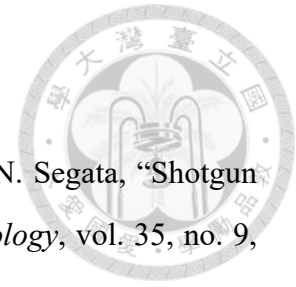
Second, pre-training is not useless: at matched 16-layer depth and 6-mer tokenization it is worth +8.94 pp at 5M reads, and near zero (slightly negative) at 50M. That is a data-efficiency result, not a ceiling result. Combined with NT-v2’s pre-training context of 2,048 tokens ($\sim 12,288$ bp) versus the 25 tokens (150 bp) used here, and with a pre-training corpus and benchmark suite aimed at regulatory genomics rather than short-read

taxonomy, the prescription is specific: keep pre-training as a mechanism, but pre-train on short microbial reads with overlapping long k -mers (or a hashed equivalent), rather than expecting a larger eukaryote-regulatory GFM to close the gap.

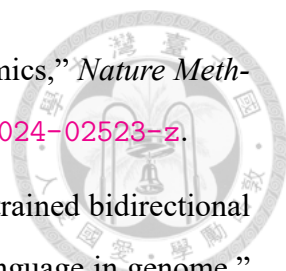


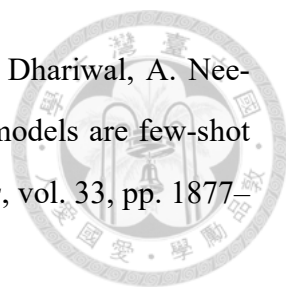
Realising that direction, evaluating metagenome-pre-trained BPE models that were out of scope here, and completing species-level scaling and real-community sample evaluation, represent the most promising path toward a competitive, generalisable GFM-based metagenomic classifier.

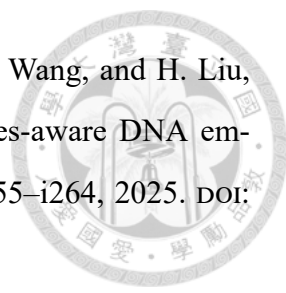
References

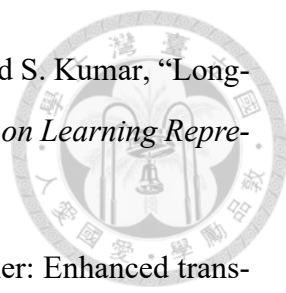


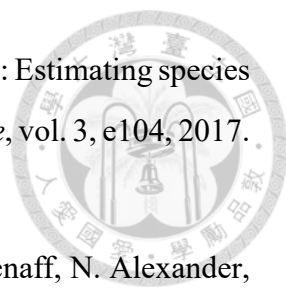
- [1] C. Quince, A. W. Walker, J. T. Simpson, N. J. Loman, and N. Segata, “Shotgun metagenomics, from sampling to analysis,” *Nature Biotechnology*, vol. 35, no. 9, pp. 833–844, 2017. DOI: [10.1038/nbt.3935](https://doi.org/10.1038/nbt.3935).
- [2] S. H. Ye, K. J. Siddle, D. J. Park, and P. C. Sabeti, “Benchmarking metagenomics tools for taxonomic classification,” *Cell*, vol. 178, no. 4, pp. 779–794, 2019. DOI: [10.1016/j.cell.2019.07.010](https://doi.org/10.1016/j.cell.2019.07.010).
- [3] F. P. Breitwieser, J. Lu, and S. L. Salzberg, “A review of methods and databases for metagenomic classification and assembly,” *Briefings in Bioinformatics*, vol. 20, no. 4, pp. 1125–1139, 2019. DOI: [10.1093/bib/bbx120](https://doi.org/10.1093/bib/bbx120).
- [4] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990. DOI: [10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2).
- [5] B. Buchfink, C. Xie, and D. H. Huson, “Fast and sensitive protein alignment using DIAMOND,” *Nature Methods*, vol. 12, no. 1, pp. 59–60, 2015. DOI: [10.1038/nmeth.3176](https://doi.org/10.1038/nmeth.3176).
- [6] D. E. Wood, J. Lu, and B. Langmead, “Improved metagenomic analysis with Kraken 2,” *Genome Biology*, vol. 20, p. 257, 2019. DOI: [10.1186/s13059-019-1891-0](https://doi.org/10.1186/s13059-019-1891-0).
- [7] D. Kim, L. Song, F. P. Breitwieser, and S. L. Salzberg, “Centrifuge: Rapid and sensitive classification of metagenomic sequences,” *Genome Research*, vol. 26, no. 12, pp. 1721–1729, 2016. DOI: [10.1101/gr.210641.116](https://doi.org/10.1101/gr.210641.116).
- [8] A. Wichmann, E. Buschong, A. Müller, D. Jünger, A. Hildebrandt, T. Hankeln, and B. Schmidt, “MetaTransformer: Deep metagenomic sequencing read classification using self-attention models,” *NAR Genomics and Bioinformatics*, vol. 5, no. 3, lqad082, 2023. DOI: [10.1093/nargab/lqad082](https://doi.org/10.1093/nargab/lqad082).
- [9] H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla, N. Lopez Carranza, A. H. Grzywaczewski, F. Oteri, C. Dallago, E. Trop, B. P. de Almeida, H. Sirelkhatim, G. Richard, M. Skwark, K. Beguir, M. Lopez, and T. Pierrot, “The Nucleotide Transformer: Build-

- 
- ing and evaluating robust foundation models for human genomics,” *Nature Methods*, 2024, Preprint on bioRxiv, 2023. DOI: [10.1038/s41592-024-02523-z](https://doi.org/10.1038/s41592-024-02523-z).
- [10] Y. Ji, Z. Zhou, H. Liu, and R. V. Davuluri, “DNABERT: Pre-trained bidirectional encoder representations from transformers model for DNA-language in genome,” *Bioinformatics*, vol. 37, no. 15, pp. 2112–2120, 2021. DOI: [10.1093/bioinformatics/btab083](https://doi.org/10.1093/bioinformatics/btab083).
- [11] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu, “DNABERT-2: Efficient foundation model and benchmark for multi-species genome,” *arXiv preprint arXiv:2306.15006*, 2024.
- [12] E. Nguyen, M. Poli, M. G. Durrant, A. W. Thomas, B. Kang, J. Sullivan, M. Y. Ng, A. Lewis, A. Patel, A. Lou, *et al.*, “Sequence modeling and design from molecular to genome scale with Evo,” *Science*, vol. 386, no. 6723, 2024. DOI: [10.1126/science.ad09336](https://doi.org/10.1126/science.ad09336).
- [13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [14] Z. Zhang, S. Schwartz, L. Wagner, and W. Miller, “A greedy algorithm for aligning DNA sequences,” *Journal of Computational Biology*, vol. 7, no. 1–2, pp. 203–214, 2000. DOI: [10.1089/10665270050081478](https://doi.org/10.1089/10665270050081478).
- [15] Q. Liang, P. W. Bible, Y. Liu, B. Zou, and L. Wei, “DeepMicrobes: Taxonomic classification for metagenomics with deep learning,” *NAR Genomics and Bioinformatics*, vol. 2, no. 1, 2020. DOI: [10.1093/nargab/lqaa009](https://doi.org/10.1093/nargab/lqaa009).
- [16] F. Mock, F. Kretschmer, A. Kriese, S. Böcker, and M. Marz, “Taxonomic classification of DNA sequences beyond sequence similarity using deep neural networks,” *Proceedings of the National Academy of Sciences*, vol. 119, no. 35, e2122636119, 2022. DOI: [10.1073/pnas.2122636119](https://doi.org/10.1073/pnas.2122636119).
- [17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

- 
- [18] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
 - [19] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smestad, A. Costa, M. Fazel-Zarandi, T. Sercu, S. Candela, and A. Rives, “Evolutionary-scale prediction of atomic-level protein structure with a language model,” *Science*, vol. 379, no. 6637, pp. 1123–1130, 2023. DOI: [10.1126/science.adc2574](https://doi.org/10.1126/science.adc2574).
 - [20] E. Nguyen, M. Poli, M. Faizi, A. Thomas, M. Wornow, C. Birch-Sykes, S. Massaroli, A. Patel, C. Rabideau, Y. Bengio, S. Ermon, C. Ré, and S. Baccus, “HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
 - [21] Y. Schiff, C.-H. Kao, A. Gokaslan, T. Dao, A. Gu, and V. Kuleshov, “Caduceus: Bi-directional equivariant long-range DNA sequence modeling,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
 - [22] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, “Hyena Hierarchy: Towards larger convolutional language models,” *arXiv preprint arXiv:2302.10866*, 2023.
 - [23] G. Brix, M. G. Durrant, J. Ku, M. Poli, G. Brockman, D. Chang, G. A. Gonzalez, S. H. King, D. B. Li, A. T. Merchant, M. Naghipourfar, E. Nguyen, C. Ricci-Tam, D. W. Romero, *et al.*, “Genome modeling and design across all domains of life with Evo 2,” *bioRxiv*, 2025. DOI: [10.1101/2025.02.18.638918](https://doi.org/10.1101/2025.02.18.638918).
 - [24] L. M. Lindsey, N. L. Pershing, A. Habib, K. Dufault-Thompson, W. Z. Stephens, A. J. Blaschke, X. Jiang, and H. Sundar, “The impact of tokenizer selection in genomic language models,” *Bioinformatics*, vol. 41, no. 9, btaf456, 2025. DOI: [10.1093/bioinformatics/btaf456](https://doi.org/10.1093/bioinformatics/btaf456).
 - [25] C. Peng, J. Shang, J. Guan, D. Wang, and Y. Sun, “ViraLM: Empowering virus discovery through the genome foundation model,” *Bioinformatics*, vol. 40, no. 12, btae704, 2024. DOI: [10.1093/bioinformatics/btae704](https://doi.org/10.1093/bioinformatics/btae704).

- 
- [26] Z. Zhou, W. Wu, H. Ho, J. Wang, L. Shi, R. V. Davuluri, Z. Wang, and H. Liu, “DNABERT-S: Pioneering species differentiation with species-aware DNA embeddings,” *Bioinformatics*, vol. 41, no. Supplement_1, pp. i255–i264, 2025. doi: [10.1093/bioinformatics/btaf188](https://doi.org/10.1093/bioinformatics/btaf188).
- [27] O. Liu, S. Jaghouar, J. Hagemann, S. Wang, J. Wiemels, J. Kaufman, and W. Neiswanger, “METAGENE-1: Metagenomic foundation model for pandemic monitoring,” *arXiv preprint arXiv:2501.02045*, 2025.
- [28] Z. Zhou, W. Wu, H. Ho, Z. Wang, *et al.*, “GenomeOcean: An efficient genome foundation model trained on large-scale metagenomic assemblies,” *bioRxiv*, 2025. doi: [10.1101/2025.01.30.635558](https://doi.org/10.1101/2025.01.30.635558).
- [29] N. Houlsby, A. Giurciu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 2790–2799.
- [30] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” *arXiv preprint arXiv:2101.00190*, 2021.
- [31] H. Zhou, A. Shrikumar, and A. Kundaje, “Towards a better understanding of reverse-complement equivariance for deep learning models in regulatory genomics,” in *Proceedings of the Machine Learning in Computational Biology (MLCB) Workshop, PMLR*, vol. 165, 2022. [Online]. Available: <https://proceedings.mlr.press/v165/zhou22a.html>.
- [32] M. Ma, “Reverse-complement consistency for DNA language models,” *arXiv preprint arXiv:2509.18529*, 2025.
- [33] V. Mallet and J.-P. Vert, “Reverse-complement equivariant networks for DNA sequences,” *bioRxiv*, 2021. doi: [10.1101/2021.06.03.446953](https://doi.org/10.1101/2021.06.03.446953).
- [34] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, “Class-balanced loss based on effective number of samples,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.

- 
- [35] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, “Long-tail learning via logit adjustment,” in *International Conference on Learning Representations (ICLR)*, 2021.
 - [36] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127 063, 2024. DOI: [10.1016/j.neucom.2023.127063](https://doi.org/10.1016/j.neucom.2023.127063).
 - [37] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira, “Perceiver: General perception with iterative attention,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 4651–4664.
 - [38] J. Lee, Y. Lee, J. Kim, A. R. Kosiosek, S. Choi, and Y. W. Teh, “Set transformer: A framework for attention-based permutation-invariant neural networks,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 3744–3753.
 - [39] A. Almeida, S. Nayfach, M. Boland, F. Strozzi, M. Beracochea, Z. J. Shi, K. S. Pollard, E. Sakharova, D. H. Parks, P. Hugenholtz, *et al.*, “A unified catalog of 204,938 reference genomes from the human gut microbiome,” *Nature Biotechnology*, vol. 39, no. 1, pp. 105–114, 2021. DOI: [10.1038/s41587-020-0603-3](https://doi.org/10.1038/s41587-020-0603-3).
 - [40] W. Huang, L. Li, J. R. Myers, and G. T. Marth, “ART: A next-generation sequencing read simulator,” *Bioinformatics*, vol. 28, no. 4, pp. 593–594, 2012. DOI: [10.1093/bioinformatics/btr708](https://doi.org/10.1093/bioinformatics/btr708).
 - [41] A. Sczyrba, P. Hofmann, P. Belmann, D. Koslicki, S. Janssen, J. Dröge, I. Gregor, S. Majda, J. Fiedler, E. Dahms, *et al.*, “Critical assessment of metagenome interpretation—a benchmark of metagenomics software,” *Nature Methods*, vol. 14, no. 11, pp. 1063–1071, 2017. DOI: [10.1038/nmeth.4458](https://doi.org/10.1038/nmeth.4458).
 - [42] F. Meyer, A. Fritz, Z.-L. Deng, D. Koslicki, T. R. Lesker, A. Gurevich, G. Robertson, M. Alser, D. Antipov, F. Beghini, *et al.*, “Critical assessment of metagenome interpretation: The second round of challenges,” *Nature Methods*, vol. 19, no. 4, pp. 429–440, 2022. DOI: [10.1038/s41592-022-01431-4](https://doi.org/10.1038/s41592-022-01431-4).

- 
- [43] J. Lu, F. P. Breitwieser, P. Thielen, and S. L. Salzberg, “Bracken: Estimating species abundance in metagenomics samples,” *PeerJ Computer Science*, vol. 3, e104, 2017. DOI: [10.7717/peerj-cs.104](https://doi.org/10.7717/peerj-cs.104).
- [44] A. B. R. McIntyre, R. Ounit, E. Afshinnkoo, R. J. Prill, E. Hénaff, N. Alexander, S. S. Minot, D. Danko, J. Foox, S. Ahsanuddin, *et al.*, “Comprehensive benchmarking and ensemble approaches for metagenomic classifiers,” *Genome Biology*, vol. 18, p. 182, 2017. DOI: [10.1186/s13059-017-1299-7](https://doi.org/10.1186/s13059-017-1299-7).
- [45] D. H. Parks, M. Chuvochina, C. Rinke, A. J. Mussig, P.-A. Chaumeil, and P. Hugenholtz, “GTDB: An ongoing census of bacterial and archaeal diversity through a phylogenetically consistent, rank normalized and complete genome-based taxonomy,” *Nucleic Acids Research*, vol. 50, no. D1, pp. D785–D794, 2022. DOI: [10.1093/nar/gkab776](https://doi.org/10.1093/nar/gkab776).
- [46] Human Microbiome Project Consortium, “Structure, function and diversity of the healthy human microbiome,” *Nature*, vol. 486, no. 7402, pp. 207–214, 2012. DOI: [10.1038/nature11234](https://doi.org/10.1038/nature11234).